

Love/Hate

User Guide
v1.10

Love/Hate

Copyright © Pixel Crushers. All rights reserved.

Table of Contents

Welcome to Love/Hate.....	4
How to Get Help.....	4
Installation.....	5
Unity 2018+ and 2D Physics.....	5
Example Scene.....	5
How Love/Hate Works.....	7
Setup Overview.....	8
Faction Database.....	8
Traits.....	9
Presets.....	9
Factions.....	10
Faction Members and Faction Manager.....	13
Factions vs. Faction Members.....	13
Faction Manager.....	14
Faction Member.....	15
Stabilize PAD.....	19
Emotion Model.....	20
Faction Member Debugger.....	22
Traits on Other Objects.....	23
LOD Manager.....	24
Deeds.....	25
Deed Template Library.....	25
Deed Reporter.....	27
Tying Deeds to Gameplay.....	27
Deed Evaluation.....	28
Automatic Character Interaction.....	29
Deed Reaction Events.....	29
Greeting Trigger.....	29
Gossip Trigger.....	30
Aura Trigger.....	30
Saving and Loading.....	32
Scripting.....	33
API Reference.....	33
Agent AI Skills.....	33
Namespace.....	33
LoveHate.....	34
FactionManager.....	34
FactionMember.....	34
DeedReporter.....	36
Event Handlers.....	37
Emotional State.....	41
Accessing Traits.....	41
Time Modes.....	41
Third Party Support.....	42
Adventure Creator Support.....	42
Visual Scripting Support.....	44
Dialogue System Support.....	46
Game Creator Support.....	49
Emerald AI Support.....	50
ICE Creature Control Support.....	51
ORK Framework & Makinom Support.....	53
PlayMaker Support.....	55
RPG Builder Support.....	58
plyGame Support.....	59
TradeSys Support.....	62
UFPS Support.....	64
Behavior Designer Support.....	65
Eliot AI Support.....	65
Quest Machine Support.....	65
Node Canvas Support.....	65
Zone Controller Support.....	65
Faction Database Templates.....	66
Simple Template.....	66
RPG Template.....	66
OCEAN Template.....	66
MegaOCEAN Template.....	67
Emotion Model Templates.....	67
Deed Evaluation Function.....	68
Spreadsheet Export/Import.....	70



WELCOME TO LOVE/HATE

Love/Hate is a relationship and personality simulator for Unity. It models characters' feelings about each other using emotional states and value-based judgment of deeds.

With Love/Hate, your characters will...

- **...have personalities and emotional states.**
Love/Hate uses powerful, customizable personality and emotional state models.
- **...maintain relationships.**
Love/Hate tracks how different characters and groups (factions) feel about each other, allowing you to define a dynamic web of social ties among your characters.
- **...judge and remember deeds.**
Characters witness deeds committed by others, judge them according to their own personality and relationships, and remember them.
- **...gossip with other characters.**
Characters share memories realistically, allowing news of actions to spread organically.

Love/Hate doesn't replace reason-based AI such as finite state machines and behavior trees, but instead works alongside them to provide interesting, character-based motivation.

Love/Hate is the product of several years of development in realtime simulation of emotions and value-based appraisal, incorporating the latest academic and industry research. Despite its sophistication, Love/Hate is very lightweight and CPU efficient at runtime. It includes complete, fully documented source code.

Love/Hate also provides integration with many third party assets, such as Hutong Games' *PlayMaker*, Opsive's *Behavior Designer* and *UFPS*, Icebox Studio's *Adventure Creator*, Squared 55's *ICE Creature Control*, Gaming Is Love's *ORK Framework*, UtopiaWorx's *Zone Controller*, and Pixel Crushers' *Dialogue System for Unity*. (These products must be purchased separately.)

How to Get Help

We're here to help! If you get stuck, have questions, or want to request a feature, please email us:

Email: support@pixelcrushers.com

INSTALLATION

Love/Hate imports into this folder structure:

Assets / Plugins / Pixel Crushers:

Common: *Shared assets used by Pixel Crushers products.*

LoveHate:

Example: *Examples of Love/Hate's features; you can safely delete this.*

Scripts: *Complete source code.*

Templates: *Starter templates for faction databases and deed templates.*

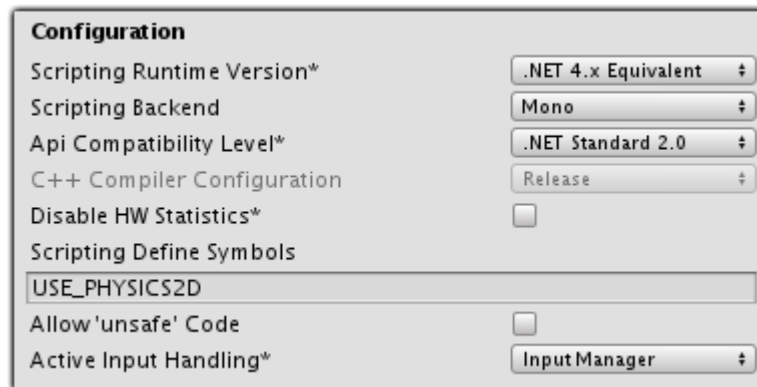
Third Party Support: *Packages to help integrate with third-party products.*

You can find the package version in Pixel Crushers/_README.txt and version release notes in Pixel Crushers/_RELEASE_NOTES.txt. Third party support packages import into Assets / Pixel Crushers.

Unity 2018+ and 2D Physics

Love/Hate supports 2D physics as well as 3D physics. In Unity 2018+, the 2D physics package (Physics2D) can be enabled or disabled, so Love/Hate's code for Unity 2018+ doesn't assume that Physics2D is available in your project. To tell Love/Hate that Physics2D is available if you're using Unity 2018+, select menu item **Tools > Pixel Crushers > Common > Misc > Enable Physics2D Support....** If you're using Unity 2017 or older, you won't see this menu item.

If you want to manually enable Physic2D support instead, select **Edit > Project Settings > Player**, and add the scripting symbol USE_PHYSICS2D as shown below:



If you're using Unity 2017, you can skip this step.

Love/Hate's example scene is in 2D. If you're using Unity 2018+, you will need to enable the Physics2D package and add enable Love/Hate support for it.

EXAMPLE SCENE



Love/Hate's example scene is in 2D, but Love/Hate supports 3D as well. If you're using Unity 2018+, please see [Unity 2018+ and 2D Physics](#) before playing the scene.

The example scene demonstrates how the player's actions affect NPCs based on their personalities and their relationships to the target of the action. You play a visitor to a tiny kingdom with a bandit problem. The scene contains four NPCs that you can interact with. When you bump into an NPC, you can see her current faction information and click buttons to perform actions such as Flatter and Insult. As you perform actions, watch the faction information window and the NPCs' animations, which are driven by Love/Hate.

The subtle halo around each NPC is her greeting range. When other characters enter it, she will greet them according to how she feels about them. If she likes the characters, she'll tell them about any of your actions that she's witnessed. This may change how those characters feel about you.

The third party support packages come with their own example scenes. You can read more about these examples in their respective sections of the manual.

HOW LOVE/HATE WORKS

Characters in Love/Hate are defined by three aspects: **relationships**, **personality**, and **emotional state**. They all use the range -100 to +100 and are affected at runtime by **deeds**.

Relationships measure the degree to which a character feels something for another character. It's possible to have multiple types of feelings (called relationship traits) between characters, such as respect, lust, and duty. By default, there is one relationship trait called *affinity* in which -100 is absolute hatred and +100 is absolute love.

In Shakespeare's *Romeo and Juliet*, Juliet might have +100 affinity to Romeo. But Tybalt, who hates Romeo's family, might have -100 affinity to Romeo.

Personality is defined by a number of personality traits that you can specify. In a simple model, you can define a single personality trait – for example, *virtue*.

In Arthurian legend, the angelic Galahad might have +100 virtue, whereas the evil usurper Mordred might have -90 virtue.

As with relationship traits, Love/Hate lets you define personality traits for your project. It includes several preconfigured templates such as OCEAN, a more sophisticated model composed of five traits. Personality is usually fairly static. For example, a ruthless, cool-headed spy typically exhibits the same personality throughout the game. However, it's possible to configure Love/Hate to automatically or manually adjust personality trait values during play.

Love/Hate also uses personality traits to describe deeds. This comes into play when characters witness deeds performed by others.

Mordred's murder of King Arthur might be -100 virtue. The wicked Morgan le Fay, whose personality aligns with the deed's, would be especially pleased to witness it. Galahad, on the other hand, would be especially displeased by the unvirtuous deed.

Emotional State is defined by three values: *Pleasure*, *Arousal*, and *Dominance* (PAD) in the ranges -100 to +100. A character's emotional state can change moment-to-moment. For example, when a character witnesses a deed that it agrees with, its Pleasure value increases. *Emotion models* allow you to summarize the three PAD values into categories. By default, Love/Hate uses a model which categorizes PAD values into eight different single-word descriptions called *temperaments*. For example, a character who is currently at Pleasure +100, Arousal +100, and Dominance +100 feels "Exuberant." A character who is currently at Pleasure -100, Arousal +100, and Dominance -100 feels "Anxious." You can use these single-word descriptions to drive your characters' behavior – for example, playing a nervous animation when the character feels "Anxious", or speaking different dialogue options based on temperament. If you want finer detail than the Temperament model's eight categories, you can use other emotion models such as the OCC model, which categorizes the three PAD values into 22 different categories.

Deeds are information about actions that occur at runtime. You'll usually define personality traits at design time and let Love/Hate update relationships and emotional state automatically at runtime in response to gameplay actions. To report deeds to Love/Hate, you'll tell Love/Hate that a character committed a deed to another character. Witnesses will then update their relationships and emotional states based on how they judge the deed. Friendly characters can also share memories of deeds (called *rumors*) as they run into each other. You can also manually adjust personality values, relationship values, and PAD values in your own scripts or visual scripting systems.

SETUP OVERVIEW

To set up Love/Hate, you'll follow these steps:

1. Create a **faction database**.
2. Configure **faction members** (characters).
3. Inform Love/Hate when **deeds** (game actions) occur.

The sections below describe these steps in detail.

FACTION DATABASE

A **faction database** is an asset that contains the following user-definable information:

The screenshot shows the 'Inspector' window for an 'Example Faction Database'. It contains four main sections: Personality Traits, Relationship Traits, Presets, and Factions. Blue arrows point from text boxes to each section, explaining their purpose.

Personality Traits (Annotation: Personality Traits are values used to define personalities)

Trait	Description
Lawfulness	How important is it to follow the law?
Boldness	How important is it to be bold?
Kindness	How important is it to be kind?

Relationship Traits (Annotation: Relationship Traits are values used to define relationships)

Trait	Description
Affinity	(Required)

Presets (Annotation: Presets are predefined values, useful as shortcuts)

Preset	Description
Citizen	Trait values for a typical citizen
Bandit	Trait values for a typical bandit

Click the double bars to the left of a preset's name to edit its traits.

Factions (Annotation: A faction is a set of personality traits and relationships assigned to a character or group)

Faction	Description	Color
Player	The player's faction	Lime
Citizens	The parent faction for citizens	Blue
Bandits	The parent faction for bandits	Red
Princess	Faction for the Princess	Blue
Cat Girl	Faction for Cat Girl, a shy citizen	Blue
Horn Girl	Faction for Horn Girl, leader of the bandits	Red
Pink Girl	Faction for Pink Girl, a bandit and sister of the Princess	Red

To create your faction database, you can copy a template or create a new, empty faction database.

Using a Template Faction Database

The **Templates** folder contains starter databases that use various psychological models such as the “OCEAN” five-factor model commonly used in behavioral psychology. To use a template, select it in the Projects view and press **Ctrl-D** (⌘-D) to duplicate it.

Using an Empty Faction Database

To create a new, empty faction database, select menu item **Assets > Create > Pixel Crushers > Love/Hate > Faction Database**.

Traits

A **trait** is a value in the range [-100,+100], where +100 means “strongly agrees” and -100 means “strongly disagrees.” Traits apply to personalities and deeds. Love/Hate lets you define the traits you want to use in your project. You can make them as streamlined or complex as you want. The Templates folder contains several starter databases with predefined traits for psychological models such as the “OCEAN” five-factor model commonly used in behavioral psychology.

Example: In the example scene, the faction database defines a trait named *Lawfulness*. The Princess, a ruler devoted to upholding the law, has a high *Lawfulness* value. Horn Girl, a Robin Hood-style bandit, has a negative *Lawfulness* value.

Traits are what characters *value*, not necessarily what they do themselves. A character might not be confident herself, but she might be attracted to confidence in others. To define a new trait, click the “+” button in the lower right of the Traits list.

Presets

A **preset** is a predefined set of traits, useful as a shortcut. You can use presets to assign default values to factions.

Example: In the example scene, the *Bandit* preset defines trait values for a typical bandit. This preset was applied to Horn Girl and Pink Girl. Then they were customized further to make each character unique. Later in the manual, you'll see that traits will also apply to deeds and other entities such as locations. You can define and apply presets to these, too.

To define a new preset, click the “+” button in the lower right of the Presets list. Enter a name and optional description. Then click the horizontal bars to the left of the preset name to edit its values.

If you want to organize presets in groups, you can use forward slash characters in the preset name.

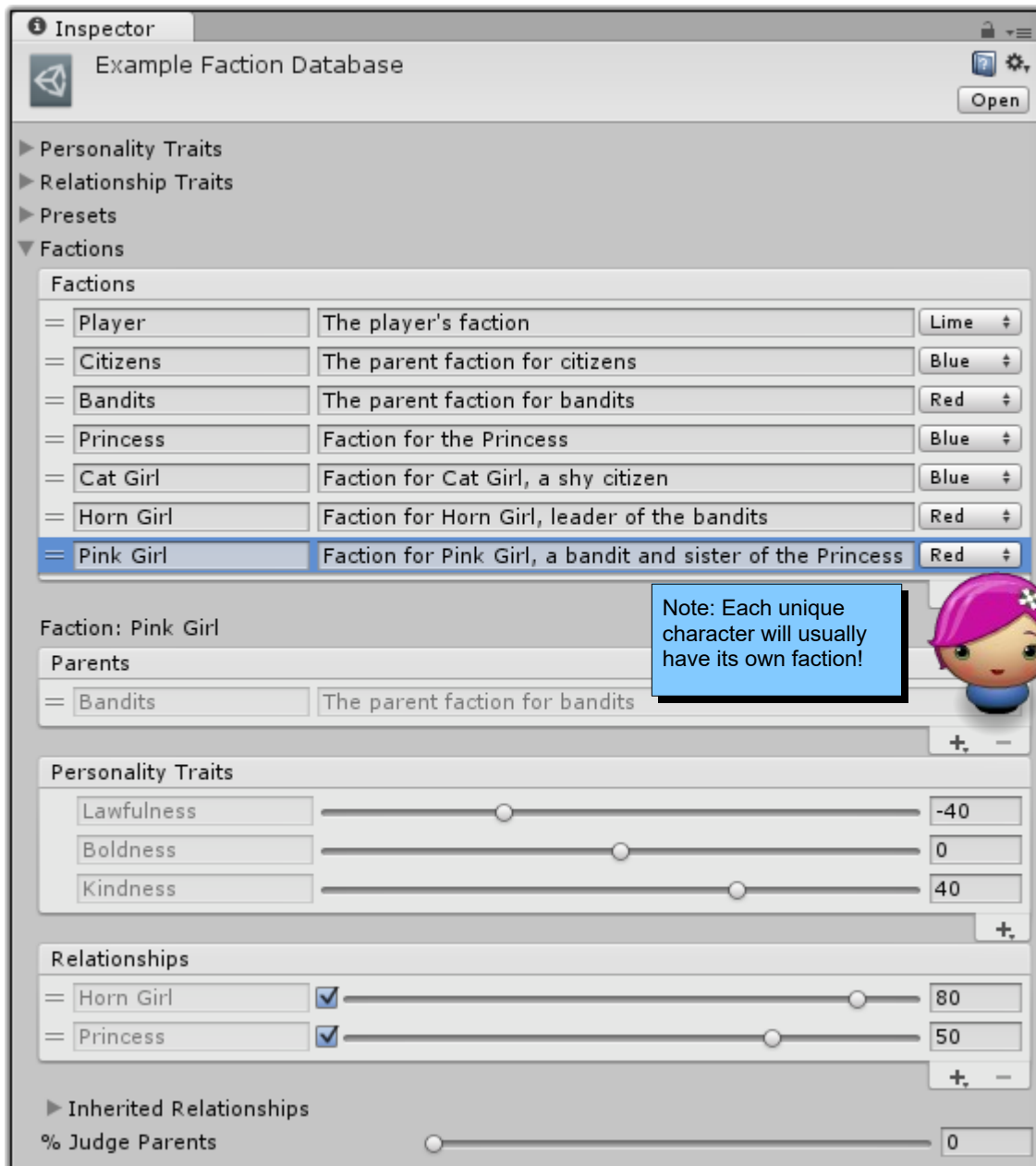
Example: Say you have several presets for monsters. You can name them *Monsters/Orc*, *Monsters/Goblin*, *Monsters/Kobold*, and so on. In faction menus, they will all be grouped in a submenu *Monsters*.

Note: If you change a preset's values, characters or deeds to which you've previously applied the preset will *not* be automatically updated with the new preset values.

Factions

A **faction** is a social circle. It defines a set of personality traits and relationships. Usually a character will have its own personal faction, although interchangeable generic NPCs, such as generic goblins, might share a common faction instead. A character who belongs to a faction is called a **faction member**.

To define a new faction, click the “+” button to the lower right of the Factions list. Enter a name, optional description, and color for this faction's gizmo in the Scene view. Then click the horizontal bars to the left of the faction's name to edit it. If you want to organize factions in groups, you can use forward slashes in their names, such as *Titans/Cronos* and *Titans/Themis*.



Inspector

Example Faction Database

Open

- ▶ Personality Traits
- ▶ Relationship Traits
- ▶ Presets
- ▼ Factions

Factions		
= Player	The player's faction	Lime
= Citizens	The parent faction for citizens	Blue
= Bandits	The parent faction for bandits	Red
= Princess	Faction for the Princess	Blue
= Cat Girl	Faction for Cat Girl, a shy citizen	Blue
= Horn Girl	Faction for Horn Girl, leader of the bandits	Red
= Pink Girl	Faction for Pink Girl, a bandit and sister of the Princess	Red

Faction: Pink Girl

Parents

= Bandits	The parent faction for bandits
-----------	--------------------------------

Personality Traits

Lawfulness	<input type="range"/>	-40
Boldness	<input type="range"/>	0
Kindness	<input type="range"/>	40

Relationships

= Horn Girl	☒	<input type="range"/>	80
= Princess	☒	<input type="range"/>	50

Inherited Relationships

% Judge Parents 0

Note: Each unique character will usually have its own faction!

Parents

A faction can belong to any number of parent factions. To add a parent, click the “+” button in the lower right of the Parents list. You only need to specify direct parents, as those parent factions may have their own parents (this faction's grandparents) and so on. Above, Pink Girl has Bandits as a parent. She inherits any relationships that Bandits have toward other factions.

Personality Traits

Use the sliders or input fields to set the faction's personality trait values. To apply a preset, click the “+” button in the lower right of the Personality Traits list. If you select “(Average from parents)”, it will average the current values of any assigned parents. If you select “(Sum from parents)”, it will add the parents' values. (Factions don't inherit these values from their parents by default.) In the screenshot on the previous page, Pink Girl's Lawfulness is -40, indicating that she has disregard for the law. Her Kindness is 40, indicating that she values kindness.

The **Trait Inheritance Type** dropdown lets you select *Average* or *Sum*. This specifies how personality traits are inherited from parents when calling `FactionDatabase.InheritPersonalityTraits()` in your scripts at runtime, or using equivalent visual scripting methods.

Relationships

Factions have relationships to other factions. Each relationship specifies a set of relationship traits in the range [-100, +100]. There will always be an **Affinity** trait, where +100 indicates absolute love and -100 indicates absolute hatred. In the screenshot on the previous page, Pink Girl has a strong +80 affinity to her idol, Horn Girl, and a moderately strong +50 affinity to her sister, the Princess.

You can choose to add more relationship traits, such as *Rivalry* or *Obligation*. You might use *Obligation* in an honor-based environment such as a traditional samurai tale. *Rivalry* makes a good counterpoint to *Affinity* in romance stories.

Example: In a sci-fi RPG, Drake, the xeno-archaeologist player character, is forced to travel with Ariana, a tough artifact smuggler. Ariana starts with low *Affinity* to Drake and high *Rivalry* – she wants to steal the artifacts; Drake wants to preserve them. As they travel the galaxy, Ariana begins to appreciate Drake's views and abilities. Her *Affinity* increases and her *Rivalry* decreases until, at some point, romance can bloom. If her *Rivalry* remains too high, however, it might override her growing affection and prevent romantic options.

Note that relationships are one-way.

Example: A third character has a secret crush on Drake. She has a high *Affinity* to him, but Drake doesn't even know that she exists, so he has no *Affinity* to her.

To add a relationship, click the “+” button in the lower right of the Relationships list.

Relationships can be inheritable (the default) or non-inheritable. To mark a relationship non-inheritable, untick the checkbox next to the subject's name.

To view relationships that are inherited from parents and ancestors, expand the **Inherited Relationships** foldout. This will display a new section containing all inherited relationships and which ancestors they're inherited from (but not all children of targets of inherited relationships since this could be an extremely big list).



The **Relationship Inheritance Type** dropdown lets you select *Average* or *Sum*. This specifies how relationships are inherited from parents.

The **% Judge Parents** value, if non-zero, specifies how much to affect relationships to the subject's parents when updating a relationship to a subject at runtime.

Export/Import Options

Spreadsheet Comma Separated Values (CSV)

If you prefer to edit your faction database values in a spreadsheet, see [Spreadsheet Export/Import](#).

JSON

You can also export and import JSON using the gear menu in the upper right of the faction database inspector.

FACTION MEMBERS AND FACTION MANAGER

Factions vs. Faction Members

Love/Hate characters are **faction members**.

It's important to understand the distinction between **factions** and **faction members**.

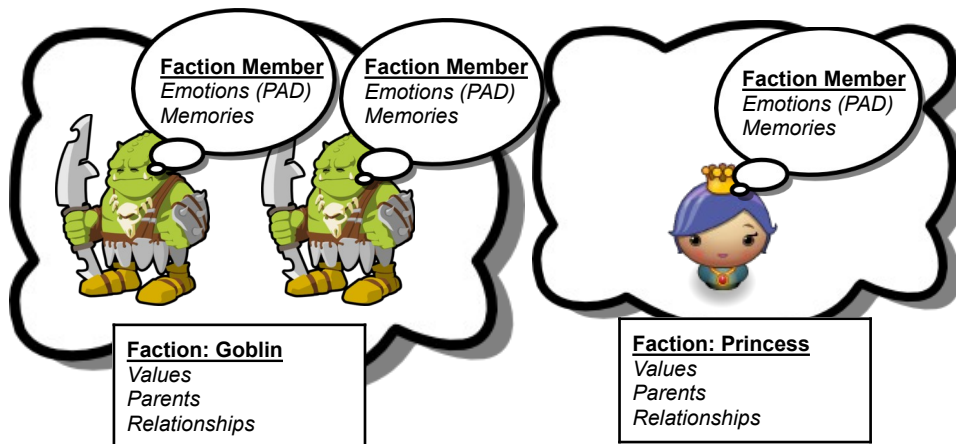
A **faction** is defined in a **faction database**. Its properties are relatively abstract. More than one faction member can belong to a faction.

A **faction member** exists on a GameObject in a scene. Its properties deal with the specific experiences of the GameObject – what it sees (witnessing deeds) and how it feels (temperament).

The “hive mind” is a classic sci-fi trope, epitomized by the Borg in *Star Trek* and the Zerg in *StarCraft*. You could think of a *faction* as the hive mind. Each *faction member* is an individual unit with its own eyes, ears, and mouth.

For unique characters, there will be a one-to-one relationship between a faction member and her unique faction. For generic characters, multiple faction members may share the same faction.

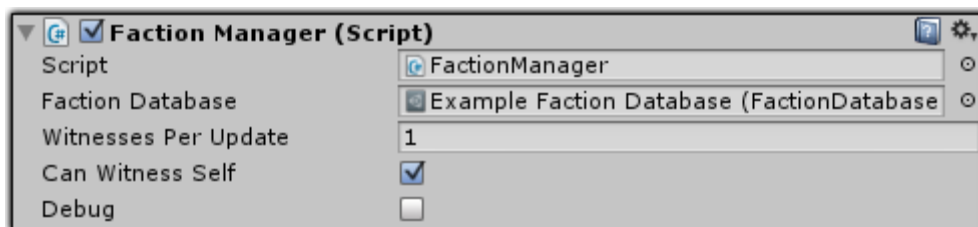
Faction	Faction Member
Abstract (Faction Database) Values, Parents, Relationships	Instantiated (GameObject) Emotions (PAD), Memories



Another way to look at it, from a technical standpoint, is that factions contain data you can define at design time, while faction members contain data such as memories and emotional states that can only be generated at runtime based on deeds.

Faction Manager

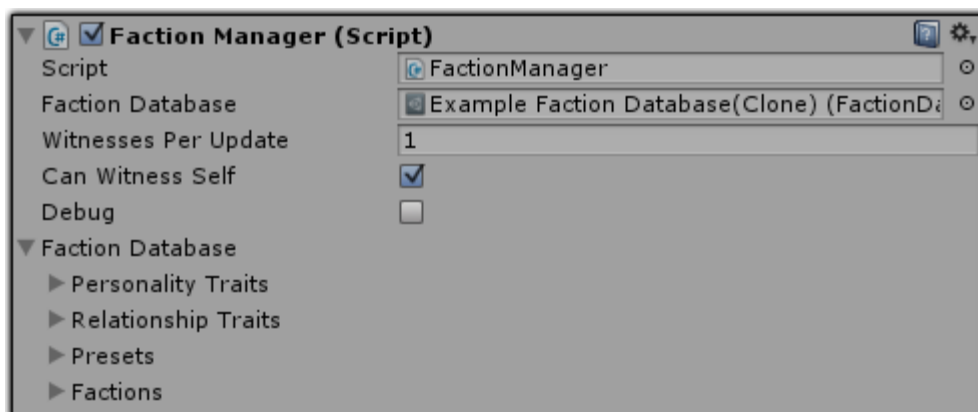
The **faction manager** helps faction members talk with each other in the scene. If your scene has any faction members, it should also have a faction manager.



To set it up a faction manager:

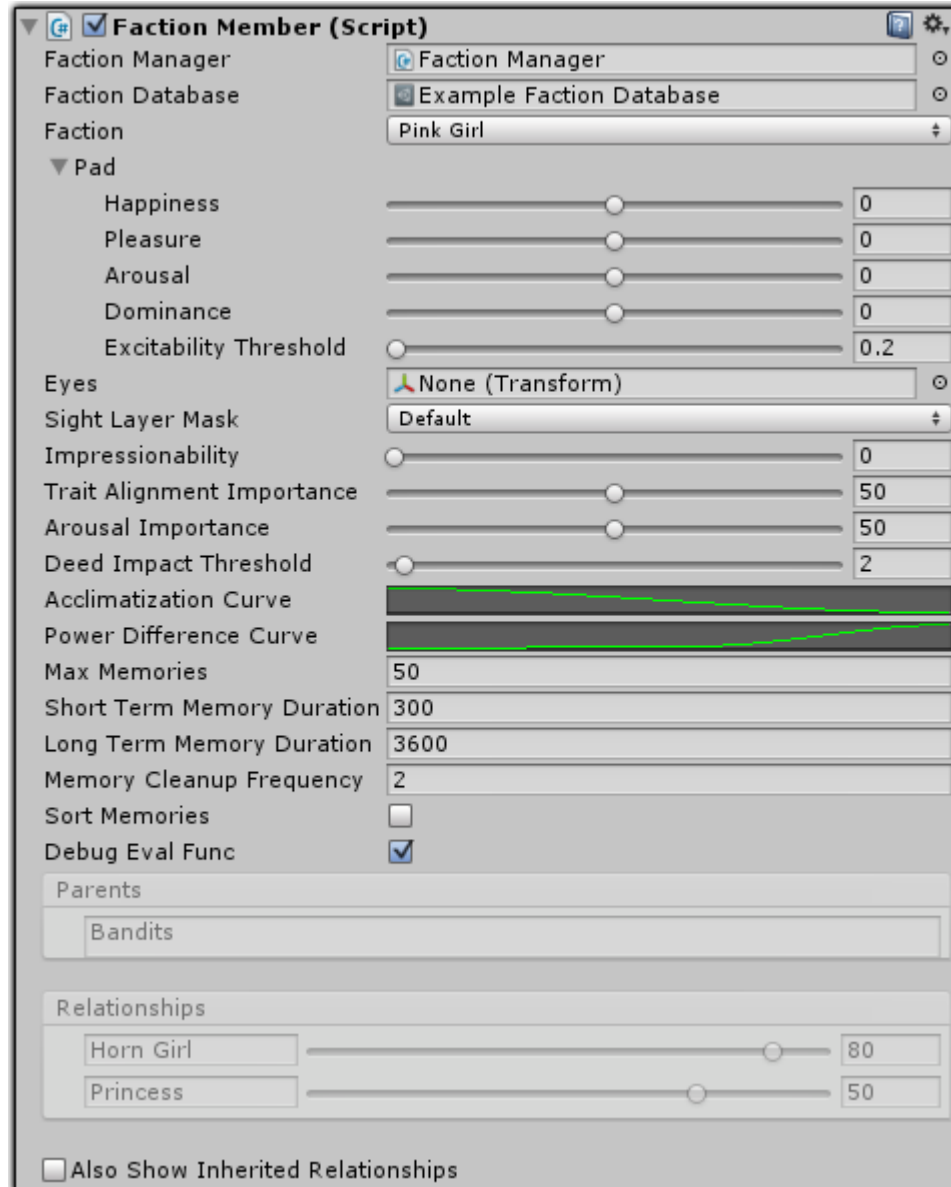
1. Select menu item **GameObject > Love/Hate > Faction Manager**. (Or just add a Faction Member component to a new, empty GameObject in your scene.)
2. Assign your faction database.
3. You can increase **Witnesses Per Update** to increase the amount of deed processing done per frame.
4. To allow faction members to witness and evaluate deeds that they themselves create, tick **Can Witness Self**.
5. If you want to log Love/Hate activity to the console, tick **Debug**.

At runtime, the Faction Manager's inspector view will also show the current state of the faction database's runtime instance:



Faction Member

A **faction member** is an NPC (a GameObject in your scene) that belongs to a faction.



To configure an NPC as a faction member, select it, and then select menu item **Component > Pixel Crushers > Love/Hate > Faction Member**.

If your scene already has a faction manager, the Faction Member script will find its faction database. Otherwise assign your faction database so you can select the faction from the dropdown menu.

Faction members share these values with all members of the faction:

Shared Faction Values

- Parents
- Values
- Relationships

In addition, each faction member stores its own data in the fields shown in the image above. Those fields are described below.

PAD (Pleasure-Arousal-Dominance)

PAD is an implementation of the Pleasure-Arousal-Dominance emotional model. (See https://en.wikipedia.org/wiki/PAD_emotional_state_model for more information about this model).

The values are in the range [-100,+100].

PAD Values

- **Pleasure (P)**: How happy or sad the character currently is.
- **Arousal (A)**: How worked up and excited the character currently is.
- **Dominance (D)**: How dominant or submissive the character currently feels.
- **Happiness**: The sum total of all pleasure values (positive and negative) over the character's lifetime.
- **Excitability Threshold**: The threshold above which the PAD values affect temperament. Temperament is an emotional state, such as *Disdainful* or *Exuberant*, based on the character's PAD values. A high excitability threshold means the character is less easily moved into an emotional state.

Temperament	P	A	D
Exuberant	+	+	+
Bored	-	-	-
Dependent	+	+	-
Disdainful	-	-	+
Relaxed	+	-	+
Anxious	-	+	-
Docile	+	-	-
Hostile	-	+	+
Neutral	0	0	0

If you want a character's PAD values to gradually return to target values over time, add an **Equalize PAD** component, which is described later in the manual.

If you want to define a more complex emotional model than Temperament, you can create an **Emotion Model** and add an **Emotional State** component to your faction member. This is described in its own section further down.

Vision

Eyes

Some deeds can only be witnessed if the NPC can see them being committed. To determine visibility, the NPC runs a raycast from the Eyes transform to the actor's origin. If Eyes is unassigned, it will use the NPC's transform, which is usually at its feet.

Sight Layer Mask

The sight layer mask specifies which layers to check when running the visibility raycast.

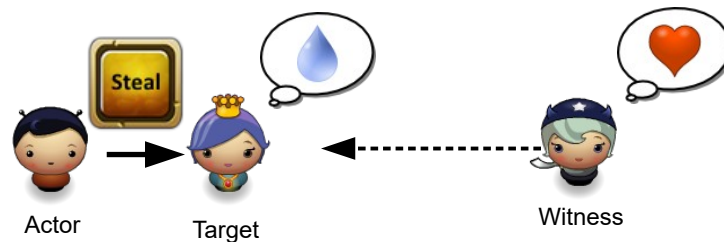
For a more sophisticated vision model, add a **Can See Advanced** component (documented at the end of this chapter).

Impressionability

Impressionability specifies how easily the faction member changes its faction's personality traits when it witnesses friends commit deeds. A faction member with high impressionability will align its personality traits more quickly with the deeds' traits.

Example: Say a street urchin befriends a thief. As the urchin witnesses the thief commit deeds such as burglary that have negative Lawfulness personality trait values, the urchin's own Lawfulness personality trait will gradually become more negative.

Memory



Characters remember **rumors**, which are subjective memories of deeds that they've witnessed or heard about. Short term memory affects the character's PAD state. Long term memory is used to gossip with other characters.

Example: If you steal from the Princess, she'll remember a negative rumor about you. Her affinity to you will decrease. It will decrease her Pleasure and increase her Arousal. If you're more powerful than she is, her Dominance will decrease. When she encounters a friend, she'll share this rumor. The friend will remember her own rumor about your deed.

If the bandit Horn Girl sees you steal from the Princess, or hears about it from a friend, she'll also remember a rumor about you. But, since she doesn't like the Princess, this rumor will improve her affinity to you and increase her Pleasure.

The values described below influence how the character subjectively evaluates the impact of a deed. They define how much importance the character places on different aspects of deeds.

Trait Alignment Importance

This value affects how much trait alignment affects the impact of a deed. When a deed's trait values exactly match the character's trait values, alignment is 100%. When the deed's trait values are in complete opposition with the character's trait values, alignment is -100%. If you set **Trait Alignment Importance** to 100, this will fully double the impact of deeds that completely match or oppose the character's traits. If you set it to 0, trait alignment will not affect the impact at all.

Deed Impact Threshold

All witnessed deeds will affect the character's PAD, but only important deeds are worth remembering. The character will only remember rumors whose perceived impact is higher than this threshold. The perceived impact of a deed is determined by how well the deed aligns with the character's values and the character's affinity to the target.

Acclimatization Curve

When characters witness repeated deeds on the same target, such as repeated flattery, the deed's impact will diminish. The acclimatization curve specifies how quickly the impact falls off. With the default value, the character will stop caring any further about the deed after 20 repetitions. However, once the deeds fall out of memory the curve is reset.

Power Difference Curve

This curve influences how the character's dominance value is affected. When a character learns about a deed that helps a friend (include the character itself), it gains dominance. When a character learns about a deed that injures a friend, it loses dominance. The Power Difference Curve helps negate dominance loss and increases dominance gain. The curve values are on a scale of 0 to 1, where 1 means an additional 100% of the dominance change is gained.

Example: The actor casts a Fear spell, which normally decreases dominance by 10. The character is power level 6. The actor is power level 1. The power difference is $6 - 1 = 5$. The character's Power Difference Curve at 5 is 0.8 (80%). 80% of 10 is 8. Since the character is more powerful, instead of taking a -10 hit to dominance, it only takes a hit of $-10 + 8 = -2$ to dominance.

Max Memories

To conserve memory, characters limit the number of things they'll remember. When memory is full, old rumors will be forgotten to make room for new. If you tick **Sort Memories**, lower-impact rumors will be forgotten first.

Short Term Memory Duration

This is the maximum amount of time that a rumor will stay in short term memory. The actual duration of each rumor will be scaled down based on the faction member's perceived importance of the rumor, which is equivalent to how much it affects the faction member's affinity to the actor. If it increases the affinity by +50 (half of the maximum +100), the rumor will stay in memory for only half the duration specified by this field.

While a rumor is in short term memory, it affects the character's PAD value.

Long Term Memory Duration

This is the maximum amount of time that a rumor will stay in long term memory, which is scaled down based on the faction member's perceived importance of the rumor.

While a rumor is in long term memory, the character can share it with friends it encounters.

Memory Cleanup Frequency

The character will check for expired memories on the frequency (in seconds) specified by this field.

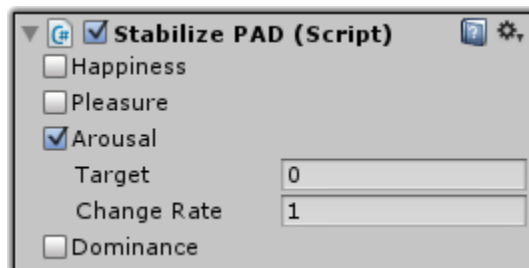
Sort Memories

If ticked, memories will be sorted in increasing impact. This makes it easier for your scripts to find the most important rumor in a character's memory. There is a slight extra overhead required to insert the memory in sorted order, but in all but the most extreme cases the overhead is negligible.

Debug Eval Func

If ticked, the faction member will log its "thought process" to the console when it evaluates a rumor. You may find this information useful to help tweak your faction database settings.

Stabilize PAD

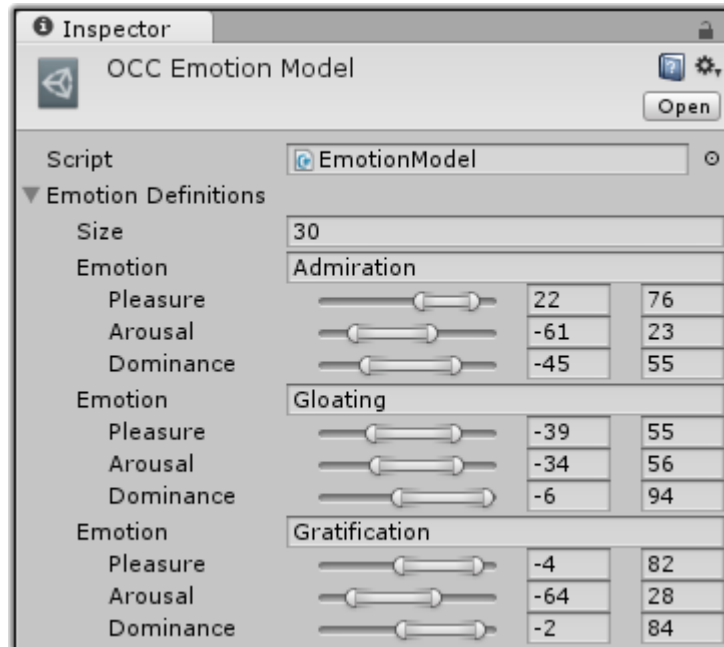


To make a faction member's PAD values gradually return to target values over time, add a **Stabilize PAD** component by selecting menu item **Component > Pixel Crushers > Love/Hate > Stabilize PAD**. Tick the values you want to affect, and then enter the target value and rate of change per second. In the screenshot above, arousal will stabilize toward zero at the rate of 1 per second.

In the screenshot above, the Stabilize PAD component will gradually bring the faction member's Arousal to 0 at a rate of 1 unit per second.

Emotion Model

If you want to define a more complex emotion model than the PAD's Temperament, you can create an Emotion Model and add an Emotional State component to your faction member(s). To create an emotion model, select menu item **Assets > Create > Pixel Crushers > Love/Hate > Emotion Model**.



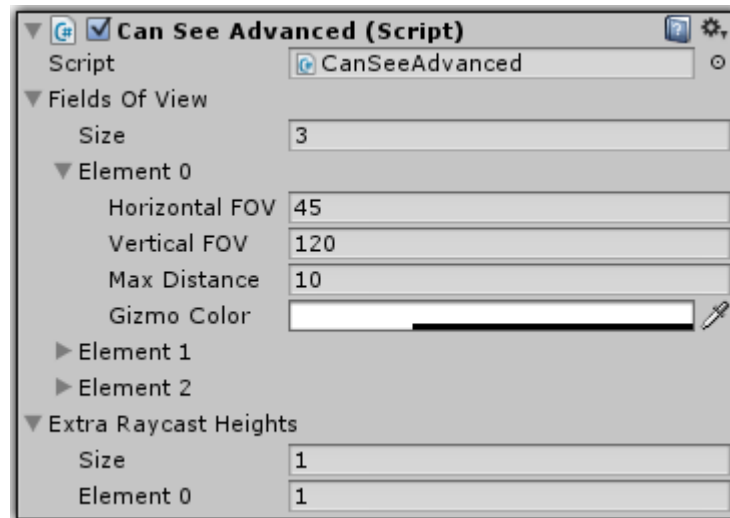
You can define any number of emotions. In the screenshot above, if a faction member's Pleasure is in the range [22,76], Arousal is [-61,23], and Dominance is [-45,55], the faction member's emotion will be *Admiration*. The Templates folder contains an emotion model that maps the 22 OCC emotions defined by behavioral psychologists.

To apply an emotion model to a faction member, add an **Emotional State** component and assign the emotion model to the **Template** field. You can customize the model for each faction member. For example, if a faction member is particularly fearful, you can increase the ranges for the *Fear* emotion.

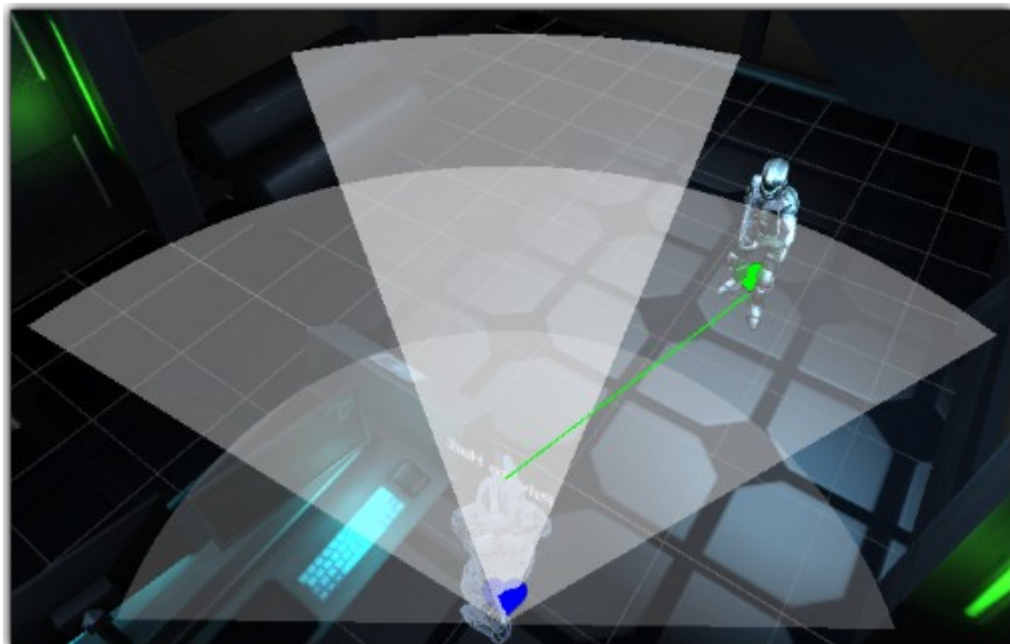
At runtime, the inspector will show the faction member's current emotional state.

Can See Advanced

To witness deeds that require a perception check, by default a faction member runs a raycast from its Eyes transform to the actor's origin. For a more sophisticated check, add a **Can See Advanced** component:

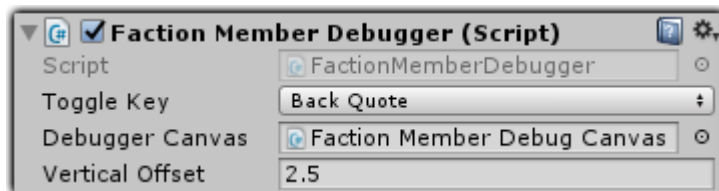


This component lets you set any number of field-of-view arcs and specify additional raycast heights to check since the default height may only check the actor's feet. The scene view gizmos show the fields of view and the result of the last perception check (green means seen, red means not seen):



Faction Member Debugger

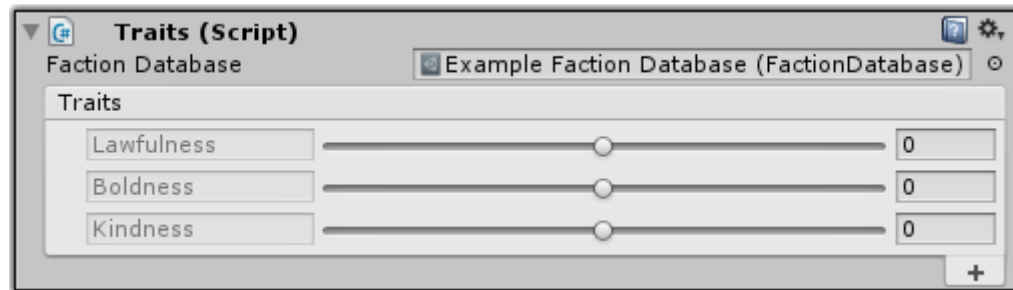
You can add a **Faction Member Debugger** component to a faction member to get an in-game display of its state.



You must assign a Faction Member Debug Canvas, which can be a GameObject or a prefab. A basic prefab is in the **Templates** folder. You can also assign a key to toggle the display on and off.

TRAITS ON OTHER OBJECTS

You can add a **Traits** component to other non-faction member GameObjects to associate traits with them. To add a traits component, select the GameObject, and then select menu item **Component > Pixel Crushers > Love/Hate > Traits**.

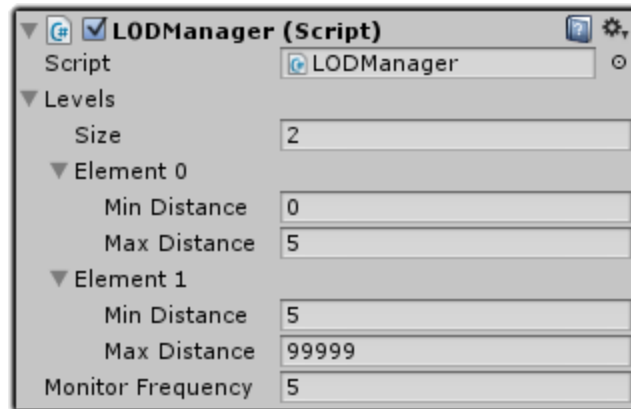


Love/Hate doesn't do anything itself with GameObjects that have Traits components. Instead, your own scripts or PlayMaker actions can examine the GameObject's traits to help make decisions. You could add traits to locations, items, even other creatures that aren't faction members.

Example: In folklore, vampires can't enter churches or cross streams. A vampire game's faction database defines a personality trait named *Purity*. In the scene, the church and stream GameObjects have Traits components with high *Purity* values. When the vampire AI script decides where to go, it can try to avoid GameObjects with a high *Purity* value.

You can use Traits in conjunction with an **Aura Trigger** (described further down in the manual) to affect characters that enter a trigger collider. The effect is based on how the character's traits align with the Aura Trigger's traits.

LOD MANAGER



The Pixel Crushers common scripts include LODManager, which is a basic level-of-detail manager.

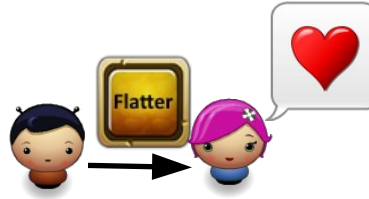
To use LODManager, add the script to your character and define the level-of-detail distances. Set Monitor Frequency to the frequency (in seconds) at which the LODManager should check distance from the player to update the current level of detail.

You can use LODManager to automatically adjust a faction member's memory cleanup frequency based on the character's distance from the player. When the player is close, the character will clean up memory according to the value of **Memory Cleanup Frequency**. When the player is far, it will reduce the frequency to conserve CPU.

To use level-of-detail in your own scripts, simply implement the `OnLOD(int level)` method:

```
public void OnLOD(int level)
{
    // Change behavior based on the value of level,
    // where level 0 is the closest to the player.
}
```

DEEDS



A **deed** is a gameplay action that NPCs can witness. It consists of:

Deed

- **Tag:** An identifying string such as “Attack” or “Flatter” used to categorize deeds
- **Actor:** The character performing the deed, usually the player
- **Target:** The character the deed is performed on
- **Impact:** A value in the range [-100,+100] indicating how harmful or beneficial the deed is
- **Aggression:** A value [-100,+100] indicating how aggressive or submissive the deed is
- **Traits:** Trait values that define the nature of the deed

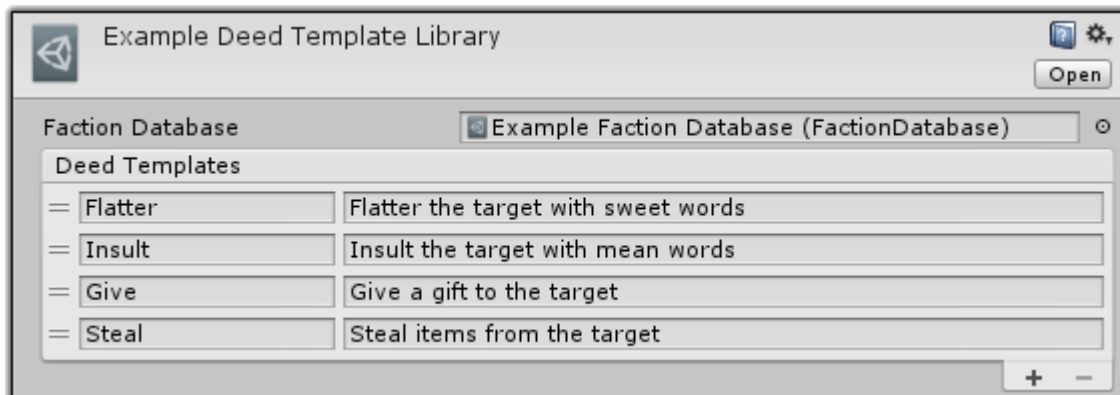
The most direct way to report a deed is to call `FactionManager.CommitDeed()` in a script. This is described in detail in the Scripting section of the manual.

An easier way to report deeds is to use a **Deed Template Library** and a **Deed Reporter**.

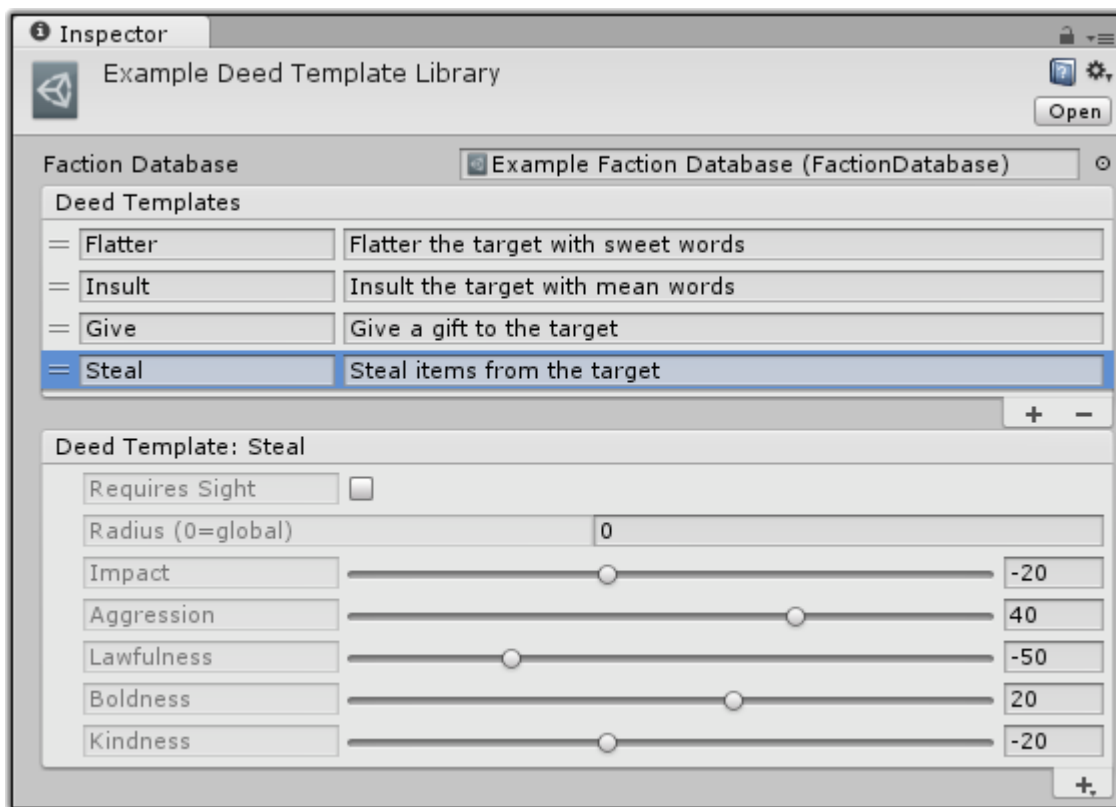
Deed Template Library

A **deed template** is an abstract definition of a deed. It defines general attributes without specifying an actor or target. Deed templates are contained in a **deed template library**.

To create a deed template library, select menu item **Assets > Create > Pixel Crushers > Love/Hate > Deed Template Library**.



To add a deed template, click the “+” button in the lower right of the Deed Templates list. Enter the deed template's tag and optional description. Then click the double bars to the left of the deed's tag to edit its attributes.



Category (Optional)

(Not shown in screenshot above.) Optional category that you own custom code can use. If you're not writing custom code to handle categories, you can leave this field blank. To create a deed category, use menu item Assets > Create > Pixel Crushers > Love/Hate > Deed Category.

Requires Sight

Some deeds can only be witnessed if the character can see them being committed. Tick this if characters should perform a visibility check to witness the deed.

Radius

This is the reporting radius of the deed. If you want all characters to be able to witness the deed, set Radius to 0.

Impact

Use the slider or edit field to set the deed's impact, where -100 is the worst possible thing that could happen to the target and +100 is the absolute best. In the example above, flattery is a minor deed, so its impact is only +5.

Aggression

Use the slider or edit the field to set the deed's aggression, where -100 is the most submissive and +100 is the most aggressive. Aggression affects characters' dominance. For example, witnessing aggressive deeds against friends will make characters feel submissive.

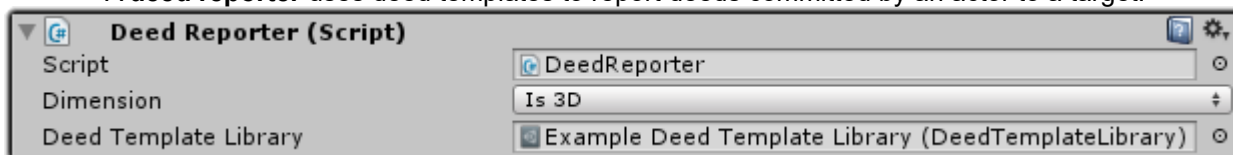
Traits

The remaining fields in the template are the trait values. Characters will compare the deed's trait values to their own trait values. The degree of alignment will modify the impact the deed has on them.

Example: The *Flatter* deed has a *Kindness* value of 70. If a character has a negative *Kindness* value, indicating that she disdains kindness, the deed will not align with her values, and the deed will have less impact on her.

Deed Reporter

A **deed reporter** uses deed templates to report deeds committed by an actor to a target.



The **DeedReporter** script goes on the character (usually the player) that will commit deeds. To add a deed reporter, select the character, and then select menu item **Component > Pixel Crushers > Love/Hate > Deed Reporter**. Then assign a deed template library. Deed reporters work in 3D and 2D scenes. Specify the type of scene in the **Dimension** dropdown.

Tying Deeds to Gameplay

When the character commits a deed, call the `ReportDeed()` method. Provide the deed template tag and the target:

```
GetComponent<DeedReporter>().ReportDeed("Flatter", pinkGirl);
```

You can also use equivalent PlayMaker actions, Dialogue System functions, etc., to implicitly call `ReportDeed()`. These are described in their own sections later in the manual.

For a more detailed example, say your player has a Combat script, and you want to report whenever the player attacks another character in the `Attack()` method. Add these lines to your script:

```
void Attack(GameObject target)
{
    animator.SetTrigger("Attack");
    target.SendMessage("TakeDamage", weaponDamage);
    var targetFactionMember = target.GetComponent<FactionMember>();
    if (targetFactionMember != null)
    {
        GetComponent<DeedReporter>.ReportDeed("Attack", targetFactionMember);
    }
}
```

(Ideally you'll cache `DeedReporter` in `Start()` rather than finding it every time the player attacks.)

Deed Evaluation

Faction members evaluate deeds using a fairly complex formula. This formula is described in detail in the **Deed Evaluation Formula** section near the end of the manual.

If the faction member's % **Judge Parents** value is non-zero, it will not only adjust its affinity to the deed actor but also to the deed actor's parents. For example, if Cat Girl does something nice, it reflects well not only on her but on her Kingdom (her parent faction). Make sure to avoid looping family trees or this will cause an infinite loop.

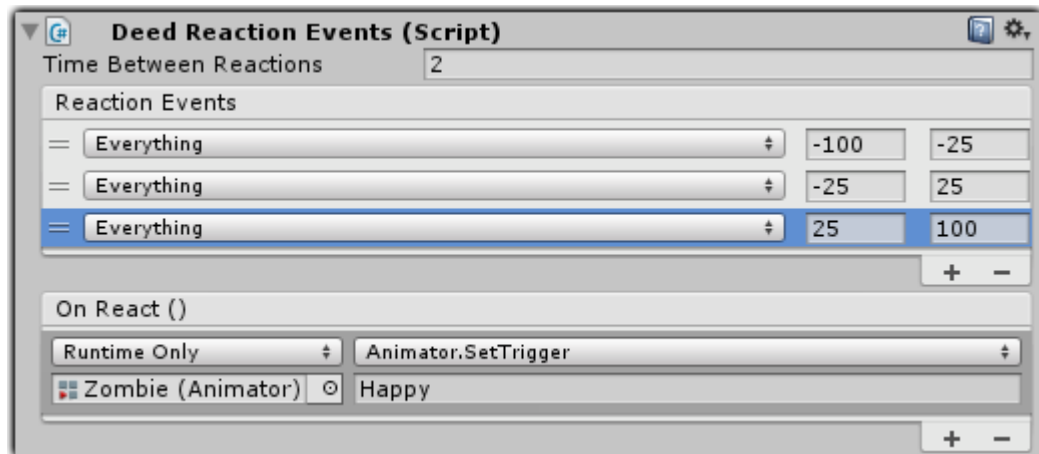
If you want a faction member to evaluate deeds differently for a specific faction (including its children), add a **Deed Evaluation Overrides** component to the faction member.

AUTOMATIC CHARACTER INTERACTION

Love/Hate provides components that allow characters can perform automatic actions based on their Love/Hate values.

Deed Reaction Events

Use a **Deed Reaction Events** component to invoke events when a character witnesses a deed.



To add a Deed Reaction Events component, select **Component > Pixel Crushers > Love/Hate > Deed Reaction Events**. To add a reaction, add an element to the Reaction Events list. Set the temperament mask (which defaults to Everything, meaning any temperament) and min and max pleasure values for the deed. Then click on the item to set the events that it should invoke. In the image above, the character is configured to set the animator trigger “Happy” when it witnesses a deed that causes it 25 to 100 pleasure.

Greeting Trigger

Use a **greeting trigger** to make a character play a greeting animation when it sees another character.



To add a greeting trigger, select the character, and then select menu item **Component > Pixel Crushers > Love/Hate > Greeting Trigger**, or **Greeting Trigger 2D** for 2D scenes. The character must have a trigger collider.

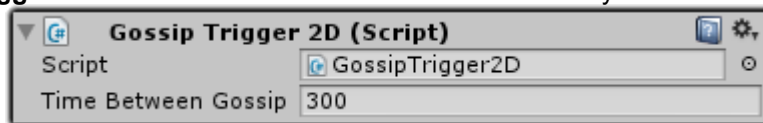
In the Animation Triggers list, specify reactions for ranges of affinity values and temperaments.

Example: In the example above, the animator trigger parameter “Sad” will trigger when the character’s affinity to the other character is less than -25. The temperament dropdown is set to *Everything* on this row, which means this line applies regardless of the character’s current temperament.

You can also add a script that implements `IGreetEventHandler` if you want the character to do something more when it greets another character. The Scripting section later in the manual explains this in more detail.

Gossip Trigger

Use a **gossip trigger** to allow a character to share rumors when they see another friendly character.



To add a gossip trigger, select the character and then select menu item **Component > Pixel Crushers > Love/Hate > Gossip Trigger** (or **Gossip Trigger 2D** for 2D scenes). The character must have a trigger collider. When another character enters the trigger, this character will check its affinity to the other character. If it’s positive, they’ll share rumors.

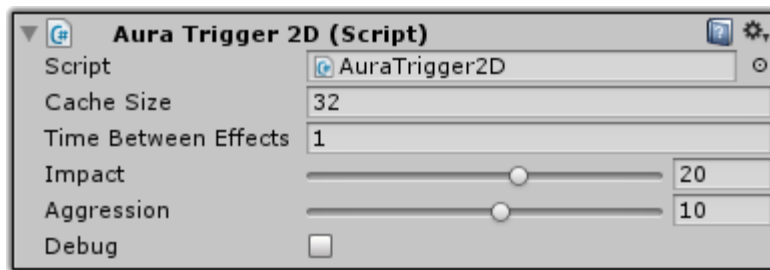
Set **Time Between Gossip** to the number of seconds the character should wait before sharing gossip again with the same friend.

To make a character play an animation when gossiping, add a Gossip Animation component: **Component > Pixel Crushers > Love/Hate > Gossip Animation**.

You can also add a custom script that implements `IGossipEventHandler` if you want the character to do something more when it gossips with another character. For example, you could play a murmuring audio clip or show a cartoon gossip bubble above their heads. The Scripting section later in the manual explains this in more detail.

Aura Trigger

Use an **aura trigger** to affect characters when they enter the trigger area.





To add an aura trigger, select a GameObject with a trigger collider and then select menu item **Component > Pixel Crushers > Love/Hate > Aura Trigger** (or **Aura Trigger 2D** for 2D scenes). This will also add a Traits component if one doesn't already exist.

Set **Time Between Effects** to the number of seconds the aura should wait before applying its effect again to the same character.

Set the values on the **Traits** component.

Impact is the strength of the aura, where a high value applies a stronger effect.

Aggression affects how submissive or dominant the aura makes the character feel based on how their traits align with the aura's traits.

Example: Say you've decided to add an aura to a vampire's lair. The Purity trait is -80, indicating it's an evil place. The Impact is 50, a powerful aura. Aggression is 30, a dominant place. When a devout villager (with a negative trait alignment) enters the lair, he will lose pleasure and dominance, and his arousal will increase. When the vampire enters its lair (with a positive trait alignment), it will gain pleasure and dominance, and its arousal will decrease.

SAVING AND LOADING

Love/Hate is designed to support easy saving and loading with third party save systems. Components don't maintain any special runtime pointers to other components or external data sources, so you can serialize and deserialize any of them without having to link anything back up.

The FactionManager and FactionMember classes provide methods to serialize and deserialize data from strings:

```
public string SerializeToString()  
public void DeserializeFromString(string s)
```

To save the state of the FactionManager or a FactionMember, call SerializeToString() and save the resulting string. To load, provide the string to DeserializeFromString().

FactionManager Serialized String Format

Serialized FactionManager data uses this format:

```
numFactions,  
(for each faction:) id,name,  
    (for each personality trait:) {personalityTraitValues}  
    numParents,{parentIDs},numRelationships,  
    (for each relationship:) {factionID,inheritable,relationshipTraitValues}
```

FactionMember Serialized String Format

Serialized FactionMember data uses this format:

```
factionID,padHappiness,padPleasure,padArousal,padDominance,numMemories,  
(for each memory:) memDeedGuid,deedTag,actorFactionID,targetFactionID,impact,  
repetitions,confidence,pleasure,arousal,dominance,shortTermTimeLeft,longTermLeft
```

Pixel Crushers Save System

Love/Hate includes the Pixel Crushers Save System, and savers for Faction Members and Faction Managers. For instructions, see the Save System documentation in Plugins / Pixel Crushers / Common / Documentation.

Other Save Systems

If you're using third party support packages such as for the Dialogue System or ORK Framework, see their sections below for instructions on using their save systems.

SCRIPTING

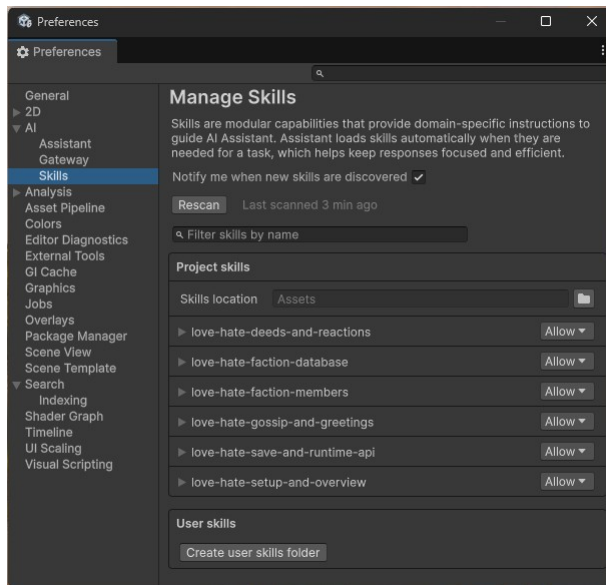
API Reference

The complete API reference is online at: <http://pixelcrushers.com/lovehate/api>

Agent AI Skills

Love/Hate is developed without generative AI.

However, if you use generative AI in your own project the files in Love/Hate's Skills folder will assist your AI agents when working with Love/Hate. If you're using Unity AI, select menu item Edit > Preferences. In the AI > Skills section, locate the love-hate-* skills and change the dropdowns from Deny to Allow:



Namespace

Love/Hate's source code is contained in the namespace `PixelCrushers.LoveHate`. Add this line to the top of your scripts to use Love/Hate:

```
using PixelCrushers.LoveHate;
```

LoveHate

You can use LoveHate static class to manage most things, such as affinities and deeds. This class serves as a wrapper for the current instance of FactionManager so you don't have to refer to FactionManager.instance directly. LoveHate and FactionManager have several useful methods. Refer to the [API reference](#) for details.

FactionManager

You will usually have one FactionManager in your scene, which you can access directly or using the LoveHate static class. However, Love/Hate also supports multiple concurrent FactionManagers. In this case, simply assign the desired FactionManager to each faction member. See *Saving and Loading* above for instructions on writing save/load code.

FactionMember

You can override default behavior on faction members by assigning your own methods to these delegates:

CanSee Delegate

If set, the faction member will call this delegate instead of the default visibility method to determine if it can see the actor of a deed.

```
void Start()
{
    GetComponent<FactionMember>().CanSee = MyCanSee;
}

public bool MyCanSee(FactionMember other, Dimension dimension)
{
    // (Return true or false)
}
```

EvaluateRumor Delegate

If set, the faction member will call this delegate instead of the default rumor evaluation method.

```
void Start()
{
    GetComponent<FactionMember>().EvaluateRumor = MyEvaluateRumor;
}

public Rumor MyEvaluateRumor(Rumor rumor, FactionMember source)
{
    // (Return a new rumor based off the source's rumor, or source's rumor.)
}
```

ShareRumor Delegate

If set, the faction member will call this delegate instead of the default rumor sharing method.

```
void Start()
{
    GetComponent<FactionMember>().ShareRumor = MyShareRumor;
}

public void MyShareRumor(Rumor rumor, FactionMember other)
{
    // Share the rumor with other.
}
```

GetPowerLevel and GetSelfPerceivedPowerLevel Delegates

There are two delegates related to power level. **GetPowerLevel** returns the faction member's power level as perceived by others. **GetSelfPerceivedPowerLevel** returns the faction member's self-perceived power level. For example, if a faction member is very insecure, he may have a lower self-perceived power level than his actual power level.

Example for GetSelfPerceivedPowerLevel:

```
void Start()
{
    var factionMember = GetComponent<FactionMember>();
    factionMember.GetSelfPerceivedPowerLevel = MySelfPerceivedLevel;
}

public float MySelfPerceivedPowerLevel()
{
    // Insecure character thinks he's half as good as he really is:
    return GetPowerLevel() / 2;
}
```

GetTraitAlignment Delegate

Assign this delegate to override the default function that computes how well a specified set of trait values aligns with the faction member's faction traits.

Example for GetTraitAlignment:

```
void Start()
{
    var factionMember = GetComponent<FactionMember>();
    factionMember.GetTraitAlignment = MyGetTraitAlignment;
}

public float MyGetTraitAlignment(float[] traits)
{
    // Add some randomness to the alignment value:
    return Traits.Alignment(faction.traits, traits) + Random.Range(-0.5f, 0.5f);
}
```

PAD

Every FactionMember has a **pad** field. You can modify its values by calling Pad.Modify().

```
void Sleep()
{
    // When sleeping, decrease arousal (3rd parameter) by 10:
    GetComponent<FactionMember>().pad.Modify(0, 0, -10, 0);
}
```

Temperament is a function of the PAD values. You can access it in code by calling PAD.GetTemperament():

```
var temperament = GetComponent<FactionMember>().pad.GetTemperament();
```

It returns an enum (see the table in the Faction Member section above) that corresponds to a specific state such as `Temperament.Exuberant` or `Temperament.Anxious`.

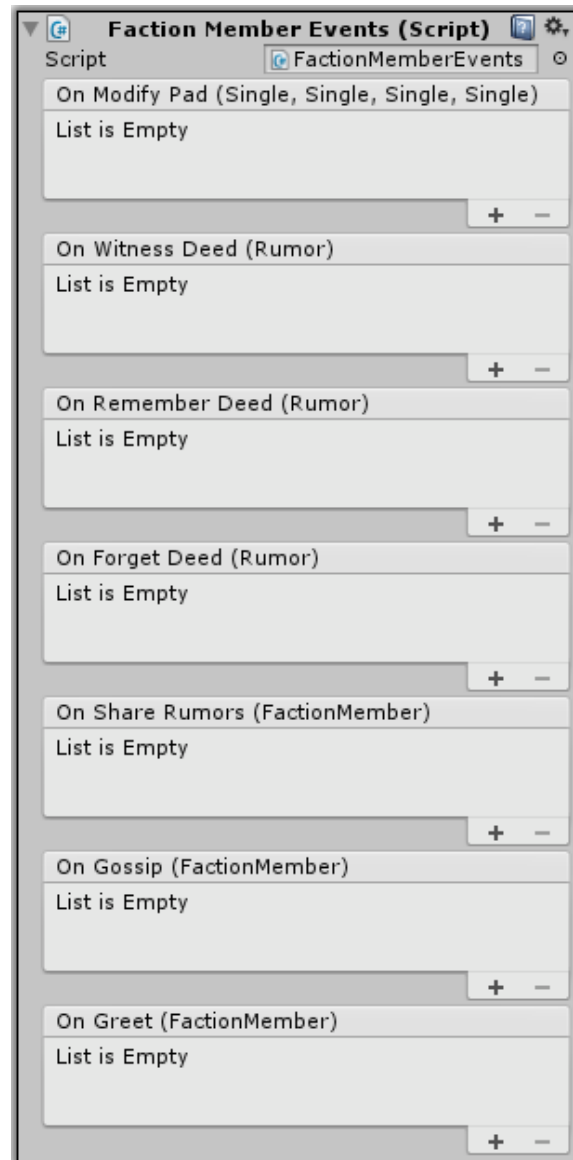
DeedReporter

DeedReporter has one method: ReportDeed(). Call it to report a deed in code. The deed tag should correspond to a deed template in the DeedReporter's deed template library.

```
GetComponent<DeedReporter>().ReportDeed("attack", selectedTarget);
```

Event Handlers

Love/Hate uses the Unity event system. If you want to visually assign your own methods to Faction Member events in Unity's Inspector view, add a Faction Member Events component (**Component > Pixel Crushers > Love/Hate > Faction Member Events**).



If you want to automatically tie your methods to Love/Hate events in script without having to assign anything in the Inspector view, add handlers for these events:

Witness Deed Event Handler

This event occurs when a faction member witnesses a deed. In the example below, if the deed displeases the faction member, the faction member will attack the actor.

```
using PixelCrushers.LoveHate;

public class MyDeedHandler : MonoBehaviour, IWitnessDeedEventHandler
{
    public void OnWitnessDeed(Rumor rumor)
    {
        if (rumor.pleasure < 0)
        {
            Say("Hey! That's my friend!");
            Attack(rumor.actorFactionID);
        }
    }
}
```

Share Rumors Event Handler

This event occurs when a faction member shares rumors with another. In the example below, the faction member will stroke its chin as if contemplating the information it just received.

```
using PixelCrushers.LoveHate;

public class MyShareRumorsHandler : MonoBehaviour, IShareRumorsEventHandler
{
    public void OnShareRumors(FactionMember other)
    {
        GetComponent<Animator>().SetTrigger("strokeChin");
    }
}
```

Gossip Event Handler

This event occurs when two characters meet and decide to share rumors. The distinction between this event and ShareRumors is that it's possible for a character to share rumors without meeting another character and gossiping – for example, if the character broadcasts news over a radio. Gossip, on the other hand, is face to face. Your custom handler can play audio, show a cartoon bubble, etc. In the example below, the faction member will play a murmur audio clip to indicate that the characters are talking.

```
using PixelCrushers.LoveHate;

public class MyGossipHandler : MonoBehaviour, IGossipEventHandler
{
    public AudioClip murmur;

    public void OnShareRumors(FactionMember other)
    {
        GetComponent<AudioSource>().PlayOneShot(murmur);
    }
}
```

Greet Event Handler

This event occurs when two characters meet and decide to greet each other. The default greet trigger already plays an animation. You can add a custom handler to do something extra such as play audio, show bark text, etc. In the example below, used for a pickpocket, the faction member appraises what the other character owns. Later in the game, the pickpocket might use this information to choose a victim.

```
using PixelCrushers.DialogueSystem;

public class MyGreetHandler : MonoBehaviour, IGreetEventHandler
{
    public void OnGreet(FactionMember other)
    {
        RememberValuables(other);
    }
}
```

Aura Event Handler

This event occurs when a character enters an aura trigger. It occurs on the aura. You can add a custom handler to make the aura do something extra. In the example below, when a character enters the aura, it instantiates a “Mist” prefab onto the character.

```
using PixelCrushers.DialogueSystem;

public class MyAuraHandler : MonoBehaviour, IAuraEventHandler
{
    public void OnAura(FactionMember other)
    {
        Instantiate(Resources.Load("Mist"), other.transform.position,
            other.transform.rotation);
    }
}
```

Enter Aura Event Handler

This event occurs when a character enters an aura trigger. It occurs on the character. You can add a custom handler to do something extra. In the example below, when a character enters a “bad” aura (one whose traits are negatively aligned with the character's traits), the character draws a weapon to prepare for trouble.

```
using PixelCrushers.DialogueSystem;

public class MyAuraHandler : MonoBehaviour, IEnterAuraEventHandler
{
    public void OnEnterAura(AbstractAuraTrigger aura)
    {
        var traits = aura.GetComponent<Traits>();
        var alignment = Traits.Alignment(traits, other.faction.traits);
        if (alignment < 0) // "Bad" aura from this character's perspective.
        {
            other.GetComponent<Combat>().DrawWeapon();
        }
    }
}
```

Emotional State

If you've added an Emotional State component to a faction member, you can call `EmotionalState.GetCurrentEmotionName()` to get the name of the current emotion, or `EmotionalState.GetCurrentEmotion()` to get the index into the `emotionDefinitions` array.

Accessing Traits

If you've added a Traits component to a non-faction member `GameObject`, you can get trait values using the `GetTraits(string traitName)` function:

```
using PixelCrushers.LoveHate;

public class VampireAI : MonoBehaviour
{
    // This function checks if an object is something the vampire should avoid:
    public bool IsObjectHarmful(GameObject obj)
    {
        var traits = obj.GetComponent<Traits>();
        return (traits != null) && (traits.GetTrait("Purity") > 50);
    }
    ... (rest of script) ...
}
```

You can also the static `Traits.Alignment(float[], float[])` function to check how well a two sets of traits align. It returns a normalized value in the range `[-1,+1]` that indicates how well their extremes match, not how close their values are. If both sets are all zero, the alignment will be zero. If all traits in both sets are at their extremes (`+100` or `-100`) and the sets match exactly, the function will return `+1`. If both sets are at their extremes and the sets are the exact opposite, the function will return `-1`.

```
// This function checks if an item is something a character would
// want based on how the item's traits align with the character's personality:
public bool DoesCharacterWantItem(FactionMember character, GameObject item)
{
    var itemTraits = item.GetComponent<Traits>().traits;
    return Traits.Alignment(character.traits, itemTraits) > 0;
}
```

Time Modes

For all time-based functions such as PAD stabilization or delays between greetings, Love/Hate uses an included class `GameTime` instead of Unity's standard `Time` class. By default, `GameTime` operates the same as `Time`. However, you can change it to realtime (i.e., not subject to time scale) or custom (in which you must manually specify the time and `deltaTime` each frame).

THIRD PARTY SUPPORT

Adventure Creator Support

Love/Hate has support for Icebox Studios' Adventure Creator. To enable it, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Adventure Creator Support.unitypackage

The package will create this folder:

Assets/Pixel Crushers/LoveHate/Third Party Support/Adventure Creator Support

Then in AC's Game Editor window > Actions section > Custom Action scripts, click "Click to set..." and specify **Assets/Pixel Crushers/LoveHate/Third Party Support/Adventure Creator Support/Scripts/Actions** to make Adventure Creator aware of the integration's actions.

Adventure Creator Example Scene

To play the Adventure Creator example scene you must first select **ManagerPackage** in the Example folder. In the Inspector view, click "Assign managers".

In the example scene, the NPCs have hotspots and interactions. To interact, click on an NPC. To keep the example simple, the interaction simply flatters the NPC.

Adventure Creator Actions

The Adventure Creator Support package will add these actions to Adventure Creator:

Action	Description
Check PAD	Checks PAD values.
Check Parent	Checks if a faction has an ancestor or direct parent.
Check Relationship	Checks the value of a relationship trait.
Check Has Personal Relationship	Checks if a faction has a direct personal relationship to another.
Create New Faction	Creates a new, empty faction.
Switch Faction	Switches a faction member to another faction.
Destroy Faction	Removes a faction from the faction database.
Modify PAD	Modifies PAD values.
Report Deed	Reports a deed. The actor must have a DeedReporter .
Set Parent	Adds or removes a parent from a faction.
Set Relationship	Sets a relationship trait value.
Set Relationship Inheritable	Specifies whether children inherit a relationship.
Share Rumors	Shares rumors from one faction member to another.

Remember Scripts for Saving and Loading

To save the state of the faction database, add a **Remember Faction Manager** script to the Faction Manager. To save the state of a faction member, add a **Remember Faction Member** script. Both scripts are available in the menu **Pixel Crushers > Love/Hate > Third Party > Adventure Creator**.

Visual Scripting Support

Love/Hate has built-in support for the Visual Scripting package. To enable it, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Visual Scripting Support.unitypackage

The package will create this folder:

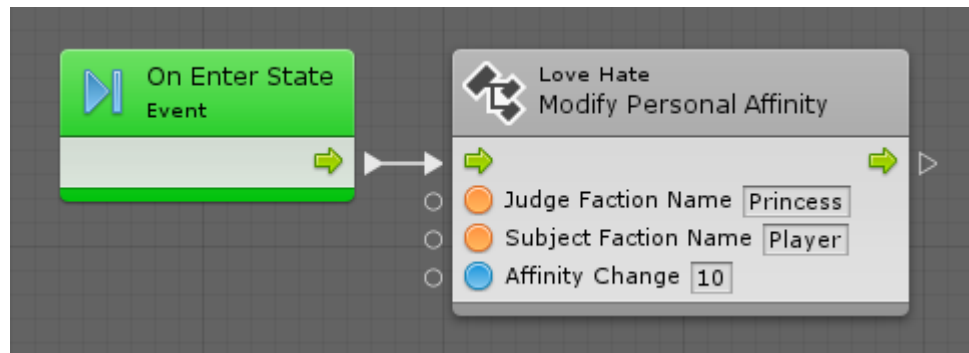
Assets/Pixel Crushers/LoveHate/Third Party Support/Visual Scripting Support

The Visual Scripting package works differently from other visual scripting systems. In other visual scripting systems, third party integrations typically provide a set of scripts that extend the system's library of actions (e.g., Report Deed action, Modify Affinity action, etc.). Visual Scripting works directly with the third party asset. You can find these actions using the Visual Scripting graph window's fuzzy finder or by right-clicking and selecting Add Unity.

These are common types to add to Visual Scripting:

Type	Reference	Gives Access To
LoveHate	PixelCrushers.LoveHate.LoveHate	Work with factions & relationships. Report deeds manually.
Deed	PixelCrushers.LoveHate.Deed	Deed record passed to LoveHate.CommitDeed.
DeedReporter	PixelCrushers.LoveHate.DeedReporter	Report deeds using a Deed Template Library.
FactionMember	PixelCrushers.LoveHate.FactionMember	Work directly with a faction member.
SaveSystem	PixelCrushers.SaveSystem	Save & load games; change scenes.

Once added, you can use them in Visual Scripting. The example screenshot below uses LoveHate.ModifyPersonalAffinity to increase the Princess's affinity to the Player by 10:



If you want Visual Scripting to react to Love/Hate events messages such as OnWitnessDeed, add a **Love Hate Visual Scripting Custom Events** component to the GameObject that will receive the event.

Event	Arguments	When It Occurs
OnWitnessDeed	Rumor	When a faction member witnesses a deed.
OnRememberDeed	Rumor	When a faction member commits a deed to memory, either by witnessing it or learning it from gossip.
OnModifyPad	PadChanges: (int) Δ happiness, (int) Δ pleasure, (int) Δ arousal, (int) Δ dominance	When a faction member's PAD changes. (Receives a struct.)
OnGreet	FactionMember	When a faction member greets another using a Greet Trigger.
OnGossip	FactionMember	When a faction member gossips with another.
OnAura	FactionMember	When a faction member enters an Aura.
OnEnterAura	AuraTrigger	When a faction member enters an Aura (occurs on Aura).

Dialogue System Support

Love/Hate has built-in Dialogue System for Unity support. To enable it, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Dialogue System Support.unitypackage

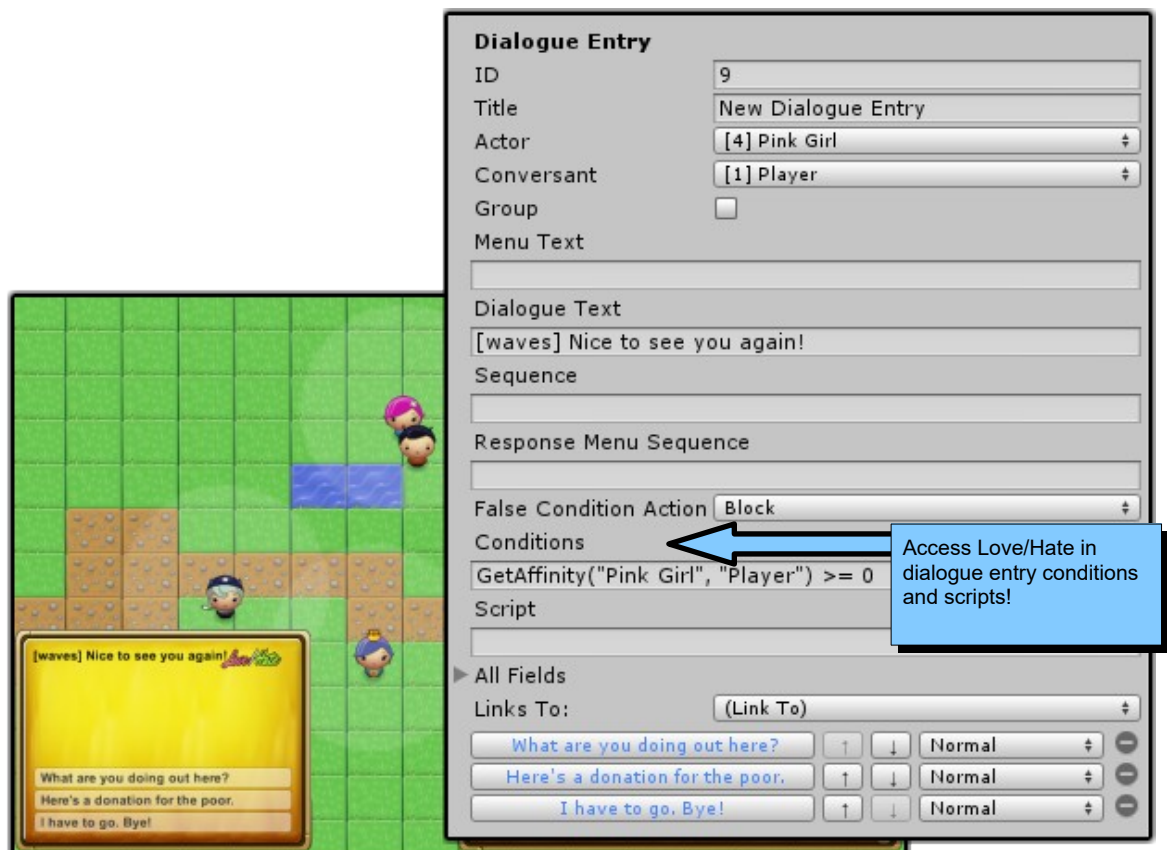
The package will create this folder:

Assets/Dialogue System/Third Party Support/LoveHate

If you've imported the Love/Hate and Dialogue System assembly definition files, also import Assets / Plugins/Pixel Crushers/Dialogue System/Third Party Support/LoveHateSupport/Scripts/LoveHateDialogueSystemAssemblyDefinitions.unitypackage.

Dialogue System Example Scene

The Dialogue System example scene plays just like the original Love/Hate example scene with the addition of a **Talk** button. When you bump into an NPC, click this button to converse.



It also adds **Save** and **Load** buttons on the upper right that demonstrate saving and loading Love/Hate data.



Love/Hate Dialogue Actor

Use the LoveHateDialogueActor component in place of the Dialogue System's DialogueActor component to keep a character's faction and Dialogue System actor in sync.

Lua Functions

The Dialogue System interfaces with Love/Hate through a library of Lua functions. To enable the Lua functions, add a **LoveHateLua** component to the Dialogue Manager GameObject (**Component > Pixel Crushers > Dialogue System > Third Party > Love/Hate > LoveHateLua**).

The **LoveHateLua** component adds the Lua functions detailed below, which you can use in conversations' Conditions and Script fields. For examples that use these Lua functions, examine the example scene's dialogue database

GetFactionName(gameObjectName) *[returns string]*

Gets the faction name of a GameObject that has a faction member component.

GetPersonalityTrait(gameObjectName, traitName) *[returns number]*

SetPersonalityTrait(gameObjectName, traitName, value)

Gets or sets a faction's personality trait value. Specify the GameObject of a faction member.

GetAffinity(judgeName, subjectName) *[returns number]*

SetAffinity(judgeName, subjectName, value)

ModifyAffinity(judgeName, subjectName, dValue)

Gets, sets, or modifies (increments/decrements) the affinity a judge feels toward a subject. For the judge and subject, provide the names of the GameObjects that have faction member components.

GetRelationshipTrait(judgeName, subjectName, traitName) *[returns number]*

SetRelationshipTrait(judgeName, subjectName, traitName, value)

ModifyRelationshipTrait(judgeName, subjectName, traitName, dValue)

Gets, sets, or modifies a relationship trait value a judge feels toward a subject. For the judge and subject, provide the names of the GameObjects that have faction member components.

SetRelationshipInheritable(judgeName, subjectName, boolean)

Specifies whether children inherit a relationship.

GetHappiness(gameObjectName) *[returns number]*

GetPleasure(gameObjectName) *[returns number]*

GetArousal(gameObjectName) *[returns number]*

GetDominance(gameObjectName) *[returns number]*

ModifyPAD(gameObjectName, dHappiness, dPleasure, dArousal, dDominance)

GetTemperament(gameObjectName) *[returns string]*

GetEmotionalState(gameObjectName) *[returns string]*

Gets or modifies (increments/decrements) the PAD values and/or emotional state of a faction member. Provide the name of the GameObject that has a faction member component.

KnowsDeed(nameString, actorNameString, targetNameString, deedTag) *[return Boolean]*

Checks if a GameObject (nameString) with a faction member component knows about a deed committed by an actor to a target. The value of actorNameString may match a DialogueActor, GameObject name, or Faction name.

ReportDeed(actorNameString, targetNameString, deedTag)

Reports a deed committed by an actor to a target. The actor must have a **Deed Reporter**.

ShareRumors(string actorName, string targetName)
Shares rumors from an actor to a target (one-way).

Example: Branching Conversation Based on Affinity

It's very easy to branch the conversation based on affinity. In the example scene, the Princess says, "You're not welcome here." if she dislikes the player. This is done by adding a simple **GetAffinity()** condition to the **Conditions** field:

The screenshot shows the 'LoveHate Dialogue Database' interface. On the left, the 'Inspector' panel displays the configuration for a 'Dialogue Entry' with ID 1. The 'Title' is 'Princess Dislikes Player', the 'Actor' is '[S] Princess', and the 'Conversant' is '[I] Player'. The 'Dialogue Text' is 'You're not welcome here.' The 'Conditions' field contains the script `GetAffinity("Princess", "Player") < 0`. Below this, the 'All Fields' section shows a list of links to other dialogue entries, each with a 'Link To' field and a 'Normal' button.

On the right, the 'Dialogue' panel shows a branching conversation tree. The tree starts with a 'START' node, which leads to a node with the text 'You're not welcome here.' This node is highlighted with a blue callout box stating: 'The Princess only speaks this entry if she dislikes the player.' From this node, the tree branches into three paths: 'You have a lovely kingdom.', 'This place is a dump! {scr', and 'Here are the bandits' plan'. Each of these paths leads to a 'Thank you, Your words are m...' node, which then leads to a 'How rude!' node, and finally to a 'Thank you, kind traveler.M' node.

A blue callout box points to the 'Conditions' field in the Inspector panel, stating: 'This is the condition on the dialogue entry.'

The condition:

```
GetAffinity("Princess", "Player") < 0
```

is only true if the Princess has a negative affinity to the player.

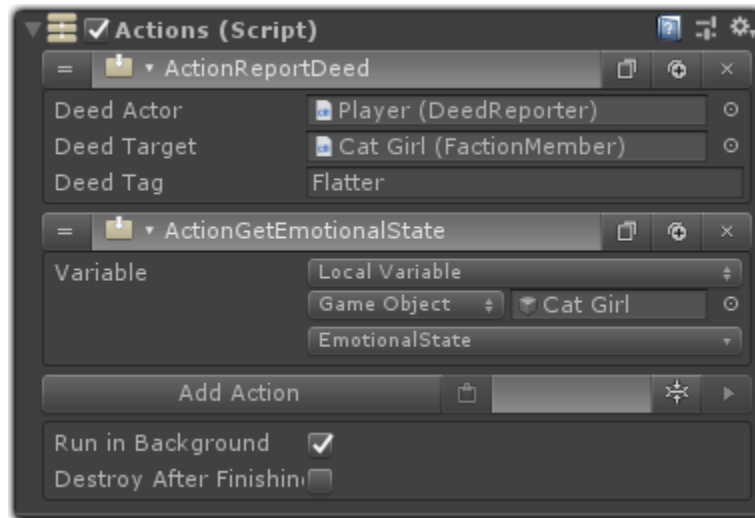
Similarly, you can use the **SetAffinity()**, **ModifyAffinity()**, and **ReportDeed()** functions in a dialogue entry's **Script** field.

Using the Dialogue System to Save and Load

To tie Love/Hate into the Dialogue System's Save System, add a **Persistent Faction Manager** component to the faction manager (**Component > Pixel Crushers > Dialogue System > Third Party > Love/Hate > Save System > Persistent Faction Manager**).

Also add a **Persistent Faction Member** component to each faction member (**Pixel Crushers > Dialogue System > Third Party > Love/Hate > Save System > Persistent Faction Member**).

Game Creator Support



To enable Game Creator support, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Game Creator Support.unitypackage

This will make the Love/Hate Game Creator module available in Game Creator's Module Manager. To enable the module, select menu item Game Creator > Module Manager. Then select the Love/Hate module and click Enable. You will be able to find the module's files in:

Assets/Plugins/GameCreator LoveHate Module

It contains an example scene and a library of Game Creator actions, conditions, and save game components.

Actions & Conditions

Love/Hate's Game Creator actions and conditions work similarly to Game Creator's built-in actions and conditions. Most of them depend on an assigned target. For example, the Modify PAD action requires that you assign a faction member's GameObject as the target. This faction member's PAD will be modified.

Saved Games

To include your faction manager and faction members in Game Creator-managed saved games, add a "Save Faction Manager to Game Creator" component to your Faction Manager GameObject and "Save Faction Member To Game Creator" components to your Faction Members.

Emerald AI Support

Love/Hate has support for Emerald AI 2024 and 3.x. To enable Emerald 2024 support, import these two packages:

Assets/Plugins/Pixel Crushers/Common/Third Party Support/Emerald 2024 Support.unitypackage
Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Emerald 2024 Support.unitypackage

The packages will create two folders:

Assets/Pixel Crushers/Common/Third Party Support/Emerald 2024 Support
Assets/Pixel Crushers/LoveHate/Third Party Support/Emerald 2024 Support

If you're using Emerald AI 3.x, import these two packages instead:

Assets/Plugins/Pixel Crushers/Common/Third Party Support/Emerald AI Support.unitypackage
Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Emerald AI Support.unitypackage

The packages will create two folders:

Assets/Pixel Crushers/Common/Third Party Support/Emerald AI Support
Assets/Pixel Crushers/LoveHate/Third Party Support/Emerald AI Support

The packages adds these features:

- Report Emerald AI combat as Love/Hate deeds.
- Adjust Emerald AI temperament based on reaction to deeds.

If you are using Emerald AI 3.0 or older, add the Scripting Define Symbol `EMERALD_AI_3_0_OR_OLDER`.

Emerald AI Setup

Add **Emerald AI Love Hate Member** components to Emerald AIs. When the player attacks an AI, this component will report a Love/Hate damage and/or kill deed. The deeds must be defined in the deed template library assigned to the player's Deed Reporter.

If you're using Emerald AI's Player Health component, add an **Emerald Player Love Hate Member** component. When an AI attacks the player, this component will report a Love/Hate damage deed. The deed must be defined in the player's deed template library.

To add emotes for Love/Hate activity, add **Greeting Emote** and/or **Gossip Emote** components.

ICE Creature Control Support

Love/Hate has support for ICE Creature Control. To enable it, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/ICE Support.unitypackage

The package will create this folder:

Assets/Pixel Crushers/LoveHate/Third Party Support/ICE Support

The package adds these features:

- Report ICE combat as Love/Hate deeds.
- Synchronize ICE indicators (power, aggressiveness, stress) with Love/Hate.
- Enable or disable interactors based on Love/Hate affinity/deeds.

ICE Example Scene



The ICE Support example scene is in ICE Support / Example / LoveHate ICE Example. The scene has three factions:

- **Zombies:** Attack everything.
- **Citizens:** Flee zombies.
- **Soldiers:** Attack zombies. If the player hurts a Soldier or Citizen, Soldiers will attack the player, too.

ICE Setup

To set up support for ICE:

1. Add an **ICE To Love Hate** component to the Faction Manager.
2. Add a **Faction Member** and **Deed Reporter** to the player.
3. Add a **Faction Member** and **Love Hate Deed Damage Handler** to creatures. This reports ICE damage to the faction member as a deed.
4. Optionally add a **Deed Reaction Events** component to creatures, and configure it to react to the pleasure value of witnessed deeds. In the example scene, creatures show a heart icon when the deed pleases them and a teardrop icon when the deed displeases them.
5. Optionally add an **Enable Interactions By Affinity** component to creatures. In the scene, when soldiers learn of a deed (either witnessed or through gossip), they check their affinity to the player. If the affinity is negative, the component enables interaction #2, which is configured to attack the player.

ICE and UFPS Support

To integrate Love/Hate with ICE's support for UFPS, replace your creatures' **ICE World Damage Adapter** components with **Love Hate Deed ICE World Damage Adapter** components. This component does the same thing as ICE World Damage Adapter but also reports the deed to Love/Hate.

To see it in action, play the example scene LoveHate ICE UFPS Example. This is a simpler example with only a UFPS player and a horde of zombies. When you shoot a zombie, it will register the deed with Love/Hate.

ORK Framework & Makinom Support

Love/Hate has support for Gaming Is Love's ORK Framework and Makinom. To enable it, import:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/Makinom Support.unitypackage
Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/ORK Support.unitypackage

The package will create these folders:

Assets/Pixel Crushers/LoveHate/Third Party Support/Makinom Support
Assets/Pixel Crushers/LoveHate/Third Party Support/ORK Support

These packages add a library of nodes that you can use in your Makinom schematics, and additional components detailed below.

ORK Setup

To use Love/Hate in an ORK game:

1. Set up your faction database and deed template library as normal.
2. In your game starter scene, add a Faction Manager GameObject. Add a **Makinom Love Hate Save Data** component to it.
3. When preparing your characters (prefabs or scene objects), instead of adding a Faction Member component, add an **Ork Faction Member** component. Assign any Auto Machines to start when certain Love/Hate events occur (Witness Deed, Enter Aura, etc.).
4. Use the Report Deed node in your Makinom schematics to report deeds to Love/Hate.
5. Use the various Check nodes to check Love/Hate values.
6. Use other Love/Hate nodes to modify PAD values, change faction membership, etc.

Ork Faction Member

The **Ork Faction Member** component is a modified version of Faction Member. It adds two things:

- Uses the combatant's power level when evaluating power levels for dominance values.
- Starts Auto Machines when Love/Hate events occur.

Connect Makinom Auto Machines to any of these Love/Hate events:

- Witness Deed: Occurs when the character witnesses a Love/Hate deed.
- Share Rumors: Occurs when the character shares rumors with another.
- Enter Aura: Occurs when the character enters a Love/Hate aura trigger.
- Greet: Occurs when the character greets another using a Greet Trigger.
- Gossip: Occurs when the character gossips with another using a Gossip Trigger.

Aura Machine Starter

The **Aura Machine Starter** component starts a Makinom AutoMachine when an aura event occurs.

Love/Hate ORK Event Steps

Love/Hate's Makinom nodes are in the Love/Hate submenu. They are:

Step	Description
Add Direct Parent	Adds a direct parent to a faction.
Change Affinity	Changes a character's affinity to a faction.
Change PAD	Changes a character's PAD values.
Change Relationship Trait	Changes the value of a relationship trait a character feels for a faction.
Check Affinity	Checks a character's affinity to a faction.
Check Arousal	Checks a character's arousal value.
Check Dominance	Checks a character's dominance value.
Check Happiness	Checks a character's happiness value.
Check Has Ancestor	Checks if a character has an ancestor faction.
Check Has Direct Parent	Checks if a character has a faction as a direct parent.
Check Knows Deed	Checks if a character knows about a deed.
Check Pleasure	Checks a character's pleasure value.
Check Relationship Trait	Checks the value of a relationship trait a character feels for a faction.
Create New Faction	Creates a new, empty faction.
Destroy Faction	Permanently removes a faction from the faction database.
Get Temperament	Gets a character's temperament value (a string).
Inherit Traits	Sets a faction's traits to the values inherited from its parents.
Remove Direct Parent	Removes a direct parent from a faction.
Report Deed	Reports a deed to Love/Hate. The actor must have a DeedReporter .
Share Rumors	Shares rumors between two characters.
Switch Faction	Changes a faction member to a different faction.

Using ORK/Makinom to Save and Load

Add a **Makinom Love Hate Save Data** component to your Faction Member to include Love/Hate data in your ORK/Makinom saved games..

PlayMaker Support

Love/Hate has built-in support for Hutong Games' PlayMaker 2 and PlayMaker 1. To enable it, import these packages:

PlayMaker 2:

Assets/Plugins/Pixel Crushers/Common/Third Party Support/PlayMaker 2 Support.unitypackage
Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/PlayMaker 2 Support.unitypackage

PlayMaker 1:

Assets/Plugins/Pixel Crushers/Common/Third Party Support/PlayMaker Support.unitypackage
Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/PlayMaker Support.unitypackage

The package adds several PlayMaker actions categorized under "Love/Hate".

PlayMaker Example Scene

The PlayMaker Example scene plays just like the original Love/Hate example scene, except the Player has a PlayMaker FSM. When you bump into an NPC, the FSM shows buttons in the top left of the screen for Flatter and Insult. The FSM demonstrates how to get factions, get affinities, and commit deeds using PlayMaker.

PlayMaker Actions

The Love/Hate PlayMaker support package adds these actions to PlayMaker's Actions browser:

Action	Description
<i>Faction Membership</i>	
Add Direct Parent	Adds a direct parent to a faction.
Create New Faction	Creates a new, empty faction.
Destroy Faction	Permanently removes a faction from the faction database.
Get Faction Name	Gets the faction name of a GameObject having a FactionMember component.
Has Ancestor	Checks if a faction has another faction as an ancestor.
Has Direct Parent	Checks if a faction has another faction as a direct parent.
Inherit Traits From Parents	Sets a faction's personality traits based on its parents, summed or averaged based on the setting in the faction database.
Remove Direct Parent	Removes a direct parent from a faction.
Switch Faction	Changes the faction that a faction member belongs to.
Use Faction Manager	Sets the faction manager that the faction member should use.
<i>Relationships</i>	
Get Affinity	Gets the affinity a faction feels toward another faction.
Get Relationship Trait	Gets the value of a relationship trait that a faction feels toward another faction.
Modify Affinity	Increments or decrements the affinity a faction feels toward another faction

Modify Relationship Trait	Increments or decrements the value of a relationship trait a faction feels toward another faction
Set Affinity	Sets the affinity a faction feels toward another faction.
Set Relationship Trait	Sets the value of a relationship trait that a faction feels toward another faction.
Set Relationship Inheritability	Sets whether a relationship is inherited by children or not.
Emotional State	
Get Happiness	Gets the PAD happiness value.
Get Pleasure	Gets the PAD pleasure value.
Get Arousal	Gets the PAD arousal value.
Get Dominance	Gets the PAD dominance value.
Get Temperament	Get a string indicating the PAD temperament.
Modify PAD	Increments or decrements PAD values.
Get Emotional State	Gets the name of the current emotional state.
Deeds	
Knows Deed	Checks if a faction member knows about a deed.
Report Deed	Reports a deed committed by an actor to a target. NOTE: The actor must have a Deed Reporter .
Share Rumors	Shares rumors between two faction members.
Saving and Loading	
Serialize To String	Gets a string representation of FactionManager or FactionMember data.
Deserialize From String	Sets FactionManager or FactionMember data from a string.
Personality Traits	
Get Personality Trait	Gets a faction's personality trait value.
Set Personality Trait	Sets a faction's personality trait value.

PlayMaker Events

To allow your PlayMaker FSM to receive events from Love/Hate, add a **Love Hate Events To PlayMaker** component. Select the FSM's GameObject, then select menu item **Pixel Crushers > Love/Hate > Third Party > PlayMaker > Love/Hate Events**. Your FSM can listen for these events:

Event	Description
OnWitnessDeed	Raised when the faction member witnesses a deed. Data: RumorVariable
OnShareRumors	Raised when the faction member shares rumors. Data: GameObjectVariable: other faction member
OnGreet	Raised when the faction member greets another faction member. Data: GameObjectVariable: other faction member
OnGossip	Raised when the faction member gossips with another member. Data: GameObjectVariable: other faction member
OnEnterAura	Raised when the faction member enters an aura. Data: GameObjectVariable: aura
OnAura	Raised on the aura when a faction member enters it. Data: GameObjectVariable: faction member that entered aura

Saving

To save PlayMaker global variables, add a **PlayMaker Global Variable Saver** component to your Save System GameObject.

RPG Builder Support



To enable RPG Builder support for Love/Hate, import these 2 packages:

- **Plugins ► Pixel Crushers ► Common ► Third Party Support ► RPG Builder Support**
- **Plugins ► Pixel Crushers ► LoveHate ► Third Party Support ► RPG Builder Support**

A complete PDF manual will be in **Plugins ► Pixel Crushers ► LoveHate ► Third Party Support ► RPG Builder Support**.

plyGame Support

Love/Hate has support for PLYoung's plyGame. To enable it, import this package:

Assets/Plugins/Pixel Crushers/LoveHate/Third Party Support/plyGame Support.unitypackage

The package adds several blocks categorized under "Love/Hate".

Blocks

The Love/Hate plyGame support package adds these blocks:

Block	Description
<i>Faction Membership</i>	
Add Direct Parent	Adds a direct parent to a faction.
Create New Faction	Creates a new, empty faction.
Destroy Faction	Permanently removes a faction from the faction database.
Get Faction Name	Gets the faction name of a GameObject that has a FactionMember component.
Has Ancestor	Checks if a faction has another faction as an ancestor.
Has Direct Parent	Checks if a faction has another faction as a direct parent.
Inherit Traits From Parents	Sets a faction's personality traits based on its parents, summed or averaged based on the setting in the faction database.
Remove Direct Parent	Removes a direct parent from a faction.
Switch Faction	Changes the faction that a faction member belongs to.
<i>Relationships</i>	
Get Affinity	Gets the affinity a faction feels toward another faction.
Get Relationship Trait	Gets the value of a relationship trait that a faction feels toward another faction.
Modify Affinity	Increments or decrements the affinity a faction feels toward another faction
Modify Relationship Trait	Increments or decrements the value of a relationship trait a faction feels toward another faction
Set Affinity	Sets the affinity a faction feels toward another faction.
Set Relationship Trait	Sets the value of a relationship trait that a faction feels toward another faction.
Set Relationship Inheritability	Sets whether a relationship is inherited by children or not.
<i>Emotional State</i>	
Get Happiness	Gets the PAD happiness value.
Get Pleasure	Gets the PAD pleasure value.
Get Arousal	Gets the PAD arousal value.
Get Dominance	Gets the PAD dominance value.

Get Temperament	Get a string indicating the PAD temperament.
Modify PAD	Increments or decrements PAD values.
Get Emotional State	Gets the name of the current emotional state.
Deeds	
Knows Deed	Checks if a faction member knows about a deed.
Report Deed	Reports a deed committed by an actor to a target. NOTE: The actor must have a Deed Reporter .
Share Rumors	Shares rumors between two faction members.

plyGame Events

To allow your plyBlox to receive events from Love/Hate, add any of the event handlers found in the menu **Component > Pixel Crushers > Love/Hate > Third Party > plyGame > Event Handlers**. Your FSM can listen for these events, each of which will also set local variables. If you add an event handler component and an event state in your plyBlox, you must also define the corresponding local variables.

Event	Description
OnWitnessDeed	Raised when the faction member witnesses a deed. Data: • deedTag (string) • deedPleasure (float) • actorFactionID (int) • targetFactionID (int)
OnRemember Deed	Raised when the faction member commits a deed to memory. Data: • deedTag (string) • deedPleasure (float) • actorFactionID (int) • targetFactionID (int)
OnForgetDeed	Raised when the faction member forgets a deed. Data: • deedTag (string) • deedPleasure (float) • actorFactionID (int) • targetFactionID (int)
OnModifyPAD	Raised when a faction member's PAD values change. Data: • happinessChange (float) • pleasureChange (float) • arousalChange (float) • dominanceChange (float)
OnShareRumors	Raised when the faction member shares rumors. Data: • otherFactionID (int)
OnGreet	Raised when the faction member greets another faction member. Data: • otherFactionID (int)
OnGossip	Raised when the faction member gossips with another member. Data: • otherFactionID (int)
OnEnterAura	Raised when the faction member enters an aura. Data:

	• auraName (string) – name of the aura GameObject
OnAura	Raised on the aura when a faction member enters it. Data: • factionID (int)

Saving and Loading in plyGame

Love/Hate's plyGame support includes persistable components for faction managers and faction members.

To save the faction manager's active faction database with plyGame saved games, add a Persistable Faction Manager component to the faction manager by using menu item **Pixel Crushers > Love/Hate > Third Party > plyGame > Persistable > Persistable Faction Manager**.

To save a faction member's state with plyGame saved games, add a Persistable Faction Member component to the faction member by using menu item **Pixel Crushers > Love/Hate > Third Party > plyGame > Persistable > Persistable Faction Member**.

TradeSys Support

Love/Hate has built-in support for CallumP's TradeSys. To enable it, import this package:

Assets/Plugins.Pixel Crushers/LoveHate/Third Party Support/TradeSys Support.unitypackage

The package will create this folder:

Assets/Pixel Crushers/LoveHate/Third Party Support/TradeSys Support

Before using the TradeSys integration or playing the example scene, make sure you've added TradeSys's required tags as described in the TradeSys manual.

TradeSys Example Scene



The example scene contains four Love/Hate-enabled trade posts, two Love/Hate-enabled traders, and a player. Each post's prices for a trader (including the player) are affected by the post's affinity to the trader. As you interact with the NPCs, their changing affinity to you will influence the prices in their faction's trading posts.

As you play the scene, watch the trade posts in the Inspector view. The prices will fluctuate depending on which trader is interacting with the post. The traders and player are controlled by modified versions of the example scripts provided with TradeSys. The example scripts don't prevent traders from interacting with the same post at the same time. If one trader buys all of a good while the other trader is trying to buy the same good, TradeSys will throw a division by zero error. To avoid this in your own project, make sure only one trader at a time interacts with a post.

TradeSys Setup

Make sure you've added TradeSys's required tags.

In TradeSys, trade posts are vendors or shops that buy and sell goods. Traders move goods between posts by buying from one post and selling to another.

TradeSys has its own faction and group system. Love/Hate doesn't use TradeSys's factions; you can use them separately from Love/Hate to limit the posts that traders visit.

How to set up Love/Hate with a trade post

1. On the **Trade Post** component, tick **Settings > Custom** pricing.
2. Add a **Faction Member** component.
3. Add a **Love/Hate Trade Post** component (**Component > Pixel Crushers > Love/Hate > Third Party > TradeSys > LoveHate Trade Post**). The Love/Hate Trade Post component has a price adjustment curve. This scales the price based on the post's affinity to the trader. For example, if the post's affinity to the trader is 50, and the curve's value at 50 is 0.25, then prices will be discounted by 25%.

LoveHateTradePost exposes the following additional functionality:

- **RestoreOriginalPrices()**: Call to set the post's current prices to their original values.
- **RecordOriginalPrices()**: Call to record the post's current prices as the new originals.
- Dictionary **originalPrice[goodsName]**: This property lets you access the recorded original prices directly.
- **AdjustPricesForFaction(Faction faction, bool buyingFromFaction = false)**: Call to adjust the post's current prices for selling to a specific faction. If buyingFromFaction is true, it instead adjusts the prices for buying from a specific faction. The table below specifies how the post's affinity for the faction affects prices:

Faction	Sell to Faction	Buy from Faction
Friend	Low	Low
Enemy	High	Low

Hot to set up Love/Hate with a trader

1. Add a **Faction Member** component.
2. Add a **Love/Hate Trader** component (**Component > Pixel Crushers > Love/Hate > Third Party > TradeSys > LoveHate Trader**).
3. Before the trader interacts with a post, call the post's `LoveHateTradePost.AdjustPricesForFaction()` method to adjust the post's prices based on its affinity to the trader. The example script `TTraderAI2D.cs` demonstrates how to call this method.

How to set up Love/Hate with the player

1. Add a **Faction Member** component.
2. Before the player interacts with a post, call the post's `LoveHateTradePost.AdjustPricesForFaction()` method to adjust the post's prices based on its affinity to the player. The example script `TSPlayer2D.cs` demonstrates how to call this method.

UFPS Support

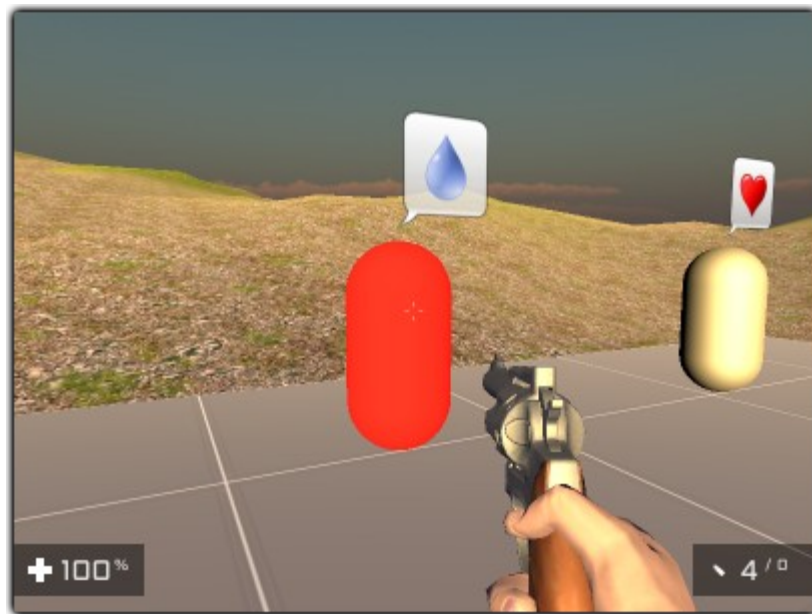
Love/Hate has built-in support for Opsive's UFPS v1. To enable it, import this package:

Assets/Plugins.Pixel Crushers/LoveHate/Third Party Support/UFPS Support.unitypackage

The package will create this folder:

Assets/Pixel Crushers/LoveHate/Third Party Support/UFPS Support

UFPS Example Scene



The UFPS Example scene contains two characters, represented by capsules: a Citizen and a Zombie, who mutually dislike each other. When you shoot one, it will be unhappy and the other will be happy.

UFPS Support Setup

To set up support for UFPS, you only need to replace your characters' **vp_DamageHandler** components with **Love Hate Deed Damage Handler** components. This component does the same thing as **vp_DamageHandler** but also reports the damage as a deed.

If you're using ICE Creature Control, follow the setup instructions in the *ICE Support* section instead.

Behavior Designer Support

Support for Behavior Designer is maintained by Opsive. You can download the support package from: <http://opsive.com/assets/BehaviorDesigner/samples.php>

Eliot AI Support

Support for Eliot AI is maintained by the author, Daniel Shevchuk. You can find more information at <http://www.eliot-ai.com/learn>

Quest Machine Support

The integration package for Quest Machine is included in Quest Machine's Third Party Support folder. The package also contains documentation on integrating Quest Machine and Love/Hate.

Node Canvas Support

Support for Node Canvas is maintained by Paradox Notion. You can download the support package from: <http://nodecanvas.paradoxnotion.com/downloads/>

Zone Controller Support

Love/Hate has built-in Zone Controller certified integration. You don't have to import any extra support packages.

FACTION DATABASE TEMPLATES

Love/Hate ships with the following faction database templates that you can use as a starting point for your own faction database.

Simple Template

The **Simple** template defines a single trait, **Virtue**. You may find that this single trait, and everything Love/Hate can do with it, is entirely sufficient to give your characters the behavior you want.=

RPG Template

The RPG template uses a small set of traits that adequately cover most situations found in typical character-based role-playing games:

Trait	Description	-100 Means	+100 Means
Charity	Kindness, empathy, mercy, and altruism	Greedy, self-centered	Altruistic, self-sacrificing
Integrity	Trustworthiness, virtuous, and scrupulous	Dishonest, sneaky	Morally upright, unwavering
Orthodoxy	Order, law, justice, and conformity	Anarchist, free-willed	Conformist, lawful
Violence	Strength and physical force	Pacifist	Warlike
Wit	Intelligence and cleverness	Simple, unintelligent, wary of intellectualism	Smart, shrewd, insightful

RPG Examples

- **Dim-witted but honorable warrior:** Integrity +80, Violence +50, Wit -80
- **Greedy, sneaky, cowardly wizard:** Charity: -90, Integrity: -80, Violence -50, Wit +80

OCEAN Template

The OCEAN template uses the OCEAN “Big Five” personality model, where +100 indicates strong alignment with the trait and -100 indicates strong disagreement with the trait.

Trait	Description
Openness	General appreciation for art, emotion adventure, and variety
Conscientiousness	Self-discipline, duty, and meeting outside expectations
Extraversion	Engagement with the external world, interacting with people
Agreeableness	Considerate, kind, generous, trusting and trustworthy, helpful
Neuroticism	Tendency to feel anger, anxiety, or depression

OCEAN Examples

- **Cool-headed spy:** Openness +70, Conscientiousness +50, Extraversion +80, Agreeableness +60, Neuroticism -90

- **Xenophobic rural bumpkin:** Openness -60, Conscientiousness +20, Extraversion -70, Agreeableness -20, Neuroticism +40

MegaOCEAN Template

The MegaOCEAN template uses a detailed subdivision of OCEAN developed by Stéphane Bura for the Game Developers Conference 2012 presentation, “Emotional AI for Expanding Worlds.” The examples below are also taken from the presentation, with values adjusted for Love/Hate’s ranges.

Openness	Conscientiousness	Extraversion	Agreeableness	Neuroticism
Imagination	Self-efficacy	Friendliness	Trust	Anxiety
Artistic Interests	Orderliness	Gregariousness	Morality	Anger
Emotionality	Dutifulness	Assertiveness	Altruism	Depression
Adventurousness	Achievement-striving	Activity Level	Cooperation	Self-consciousness
Intellect	Self-discipline	Excitement-seeking	Modesty	Immoderation
Liberalism	Cautiousness	Cheerfulness	Sympathy	Vulnerability

MegaOCEAN Examples

Faction: Shy Person. A shy person might have these personality traits.

Openness	Conscientiousness	Extraversion -80	Agreeableness	Neuroticism +20
Imagination	Self-efficacy	Friendliness	Trust	Anxiety +40
Artistic Interests	Orderliness	Gregariousness -60	Morality	Anger
Emotionality -40	Dutifulness	Assertiveness -80	Altruism	Depression
Adventurousness -80	Achievement-striving	Activity Level -20	Cooperation	Self-consciousness +80
Intellect	Self-discipline +40	Excitement-seeking -40	Modesty +80	Immoderation
Liberalism	Cautiousness +80	Cheerfulness	Sympathy	Vulnerability +40

Deed: Seduction. The deed that describes seducing someone might have these traits.

Openness +80	Conscientiousness	Extraversion +80	Agreeableness +60	Neuroticism -30
Imagination	Self-efficacy	Friendliness +40	Trust	Anxiety
Artistic Interests	Orderliness	Gregariousness +80	Morality	Anger
Emotionality +80	Dutifulness	Assertiveness +60	Altruism	Depression
Adventurousness +60	Achievement-striving	Activity Level	Cooperation	Self-consciousness
Intellect	Self-discipline +20	Excitement-seeking +60	Modesty -60	Immoderation +40
Liberalism	Cautiousness	Cheerfulness	Sympathy +40	Vulnerability

EMOTION MODEL TEMPLATES

Love/Hate ships with **OCC Emotion Model**, an emotion model template that maps the 22 OCC emotions into PAD space. The bottom of the model includes the 8 PAD temperaments as catch-alls in case the character’s current PAD values don’t fit into any of the 22 OCC emotion ranges.

DEED EVALUATION FUNCTION

This section describes how the default deed and rumor evaluation function works. You can override the default function by assigning the delegate `FactionMember.EvaluateRumor`.

A *deed* is an act committed by an actor to a target.

A *rumor* is a memory of a deed. Rumor data includes:

- **Source:** The faction member delivering the rumor
- **Action tag:** The type of act (e.g., “attack”, “heal”, etc.)
- **Actor:** The faction member who committed the act
- **Target:** The faction member to whom the act was done
- **Impact:** How harmful (-100) or beneficial (+100) the act was to the target
- **Aggression:** How aggressive this type of act is
- **Traits:** Personality trait values associated with this act
- **Confidence:** How sure the source is that the rumor is true

When a faction member learns of a deed or rumor, it affects how the faction member feels – for example, does the action make it like or dislike the actor? Does the action make it feel happy, sad, excited, submissive?

Faction members can acquire rumors by hearing them from another member or by directly witnessing a deed. When a faction member directly witnesses a deed, it evaluates the deed as if it were sharing a rumor with itself with 100% confidence.

When a faction member evaluates a rumor, it updates the following information:

- Affinity to the actor
- Pleasure, Arousal, Dominance (PAD) values
- Short-term and long-term memory

The Formula

This is how the faction member updates information:

1. Compute my confidence in the rumor, based on the source’s confidence in the rumor and how much I like (trust) the source:

$$\text{confidence} = \text{affinity-to-source} \times \text{source's-confidence-in-rumor}$$

2. Compute how it affects my relationship to the actor, based on how the deed impacts the target:

$$\Delta \text{affinity-to-actor} = \text{affinity-to-target} \times \text{impact} \times \text{confidence}$$

3. Adjust $\Delta \text{affinity-to-actor}$ by how well the deed’s traits align with my own:

$$\Delta \text{affinity-to-actor} += |\Delta \text{affinity-to-actor}| \times \text{trait-alignment} / \text{trait-alignment-importance}$$

4. Adjust $\Delta \text{affinity-to-actor}$ based on my arousal level:

$$\Delta \text{affinity-to-actor} += \Delta \text{affinity-to-actor} \times \text{arousal} \times \text{arousal-importance}$$

5. Adjust my pleasure based on $\Delta \text{affinity-to-actor}$:

$$\Delta \text{pleasure} = \Delta \text{affinity-to-actor} + (\Delta \text{affinity-to-actor} \times \text{trait-alignment})$$

6. Adjust my arousal based on $\Delta \text{affinity-to-actor}$:

$$\Delta \text{arousal} = |\Delta \text{affinity-to-actor}| \times \text{arousal-importance}$$

7. Adjust my dominance based on aggression, impact, and affinities:

$$\Delta \text{dominance} = \text{sign}(\text{rumor-impact}) \times \text{sign}(\text{affinity-to-target}) \times |\text{aggression}| \times |\Delta \text{affinity-to-actor}|$$

8. Further adjust my dominance based on the power difference curve:

$$\Delta \text{dominance} += \text{power-difference-curve} \times |\Delta \text{dominance}|$$

9. If I'm impressionable and my affinity to the actor is positive, adjust my personality traits to align more closely with the deed's traits:

$$\Delta \text{personality-traits} = \text{affinity-to-actor} \times \text{impressionability}$$

You can see these numbers in Unity's Console view by ticking the faction member's **Debug Eval Func** checkbox.

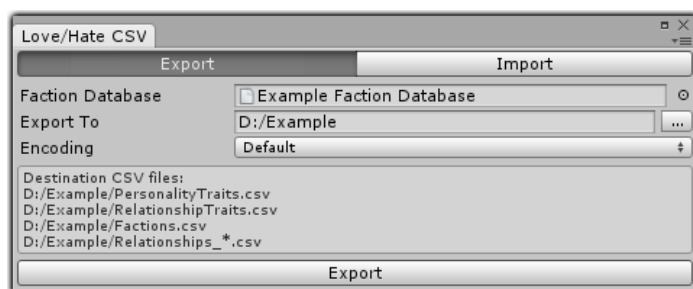
The faction member will only commit the rumor to memory if $\Delta \text{affinity-to-actor}$ is greater than its **Deed Impact Threshold**. If the rumor is a repeat (that is, it has the same actor, target, and tag), the faction member won't add a new memory; instead, it will increment the count on the existing memory and scale down the impact based on the faction member's **Deed Acclimatization Curve**.

Even if the faction member doesn't commit the rumor to memory, the rumor will still affect its emotional state and, if impressionable, its personality traits.

SPREADSHEET EXPORT/IMPORT

If you prefer to edit your faction database values in a spreadsheet, use the Love/Hate CSV window to bring . To open this window, select **Tools > Pixel Crushers > Love/Hate > CSV Export/Import...** or use the Inspector's gear menu while inspecting a faction database.

CSV Export



The **Export** button will export a set of CSV files that you can open with any spreadsheet application.

- **PersonalityTraits.csv**: The personality traits, one per row.
- **RelationshipTraits.csv**: The relationship traits, one per row.
- **Factions.csv**: All information about factions and presets except relationships, one faction/preset per row.
- **Relationships_relationship.csv**: One file per relationship trait, containing a matrix of relationship values. You only need to specify cell values for existing faction relationships. If the cell is blank or contains non-numeric text such as "none", it means there is no relationship between the factions.

When editing these CSV files, you must keep them in sync. If you add a new personality trait to PersonalityTraits.csv, you must add a corresponding column in Factions.csv.

CSV Import

Love/Hate CSV

Export Import

Faction Database

Import From

Encoding

Attempt to import these CSV files:
D:/Example/PersonalityTraits.csv
D:/Example/RelationshipTraits.csv
D:/Example/Factions.csv
D:/Example/Relationships_*.csv

Faction Spreadsheet Configuration

If you're importing from CSV files that were exported from this window, you can use the default import settings. If you're importing from a different spreadsheet format, specify the format of the Factions spreadsheet below.

Auto-assign IDs ☐

ID (A)	1
Name (B)	2
Description (C)	3
Includes Presets	☒
Preset (D)	4
Color (E)	5
% Judge Parents (F)	6
First Personality Trait (G)	7
Parents After Personality	☒

Importing will overwrite the contents of your faction database.

Reset Import

By default, the **Import** button is configured to import data in the format generated by the Export button.

However, it can also import factions from other spreadsheet layouts. You can specify the layout of your factions spreadsheet in the **Faction Spreadsheet Configuration** section. To restore the default settings, click the Reset button.

Game Data Editor note:

If you are exporting data from Stay At Home Devs' *Game Data Editor*, the importer will ignore any rows that start with "GDE_".